

# Numerical Optimization for Large Scale Problems

## Lecture Notes

Francesco Della Santa \*

Politecnico di Torino, A.Y. 2025/2026

### Abstract

Notes of the optimization laboratories held by Francesco Della Santa for the course of Numerical Optimization for Large Scale Problems.

## Contents

|          |                                             |           |
|----------|---------------------------------------------|-----------|
| <b>1</b> | <b>Direct Methods for Linear Systems</b>    | <b>5</b>  |
| 1.1      | Square Matrices . . . . .                   | 5         |
| 1.2      | Gaussian Elimination . . . . .              | 6         |
| 1.3      | Square Linear Systems . . . . .             | 6         |
| 1.4      | Condition Number . . . . .                  | 7         |
| 1.5      | Direct Methods in Matlab . . . . .          | 7         |
| 1.6      | Direct Methods in Python . . . . .          | 7         |
| 1.7      | Fill-In Phenomenon . . . . .                | 10        |
| 1.8      | Exercises . . . . .                         | 10        |
| <b>2</b> | <b>Finite Differences</b>                   | <b>12</b> |
| 2.1      | Finite Differences (Gradient) . . . . .     | 12        |
| 2.1.1    | Choice of $h$ . . . . .                     | 12        |
| 2.2      | Finite Differences (Hessian) . . . . .      | 13        |
| 2.2.1    | Choice of $h$ . . . . .                     | 13        |
| 2.3      | Finite Differences (Jacobian) . . . . .     | 13        |
| 2.4      | Exercises . . . . .                         | 13        |
| <b>3</b> | <b>Iterative Methods for Linear Systems</b> | <b>15</b> |
| 3.1      | The Gradient Method . . . . .               | 16        |
| 3.1.1    | Sequence of the Gradient Method . . . . .   | 17        |
| 3.1.2    | Convergence Properties . . . . .            | 18        |
| 3.2      | Conjugate Gradient Method . . . . .         | 18        |
| 3.2.1    | Preconditioning . . . . .                   | 19        |
| 3.3      | Other Iterative Methods . . . . .           | 20        |
| 3.4      | Exercises . . . . .                         | 20        |
| <b>4</b> | <b>Steepest Descent</b>                     | <b>22</b> |
| 4.1      | Exercises . . . . .                         | 22        |

---

\*Dipartimento di Scienze Matematiche, Politecnico di Torino, Turin, Italy

|           |                                                                        |           |
|-----------|------------------------------------------------------------------------|-----------|
| <b>5</b>  | <b>Steepest Descent with Backtracking</b>                              | <b>24</b> |
| 5.1       | Backtracking . . . . .                                                 | 24        |
| 5.2       | Exercises . . . . .                                                    | 25        |
| <b>6</b>  | <b>Newton Method with Backtracking</b>                                 | <b>27</b> |
| 6.1       | Recalling the Motivations of Newton . . . . .                          | 27        |
| 6.2       | Backtracking and Newton . . . . .                                      | 27        |
| 6.3       | Exercises . . . . .                                                    | 28        |
| <b>7</b>  | <b>Inexact Newton and Orders of Convergence</b>                        | <b>30</b> |
| 7.1       | Inexact Descent Direction and Forcing Terms . . . . .                  | 30        |
| 7.2       | Orders of Convergence . . . . .                                        | 30        |
| 7.3       | Exercises . . . . .                                                    | 31        |
| <b>8</b>  | <b>Newton and Inexact Newton Methods for Nonlinear Equations</b>       | <b>33</b> |
| 8.1       | Newton-Tangent Method and Optimization of the Merit Function . . . . . | 33        |
| 8.2       | Inexact Newton-Tangent Method for Nonlinear Equations . . . . .        | 34        |
| 8.2.1     | Inexact Descent Direction and Forcing Terms . . . . .                  | 34        |
| 8.3       | Test Functions . . . . .                                               | 34        |
| 8.4       | Exercises . . . . .                                                    | 35        |
| <b>9</b>  | <b>Gauss-Newton for Nonlinear Least Squares</b>                        | <b>37</b> |
| 9.1       | Gauss-Newton and Linear Least Squares . . . . .                        | 37        |
| 9.2       | Least Squares and Regression in Machine Learning . . . . .             | 38        |
| 9.2.1     | Linear Least Squares and Linear Regression . . . . .                   | 38        |
| 9.2.2     | Gauss-Newton and Nonlinear Regression . . . . .                        | 38        |
| 9.3       | Test Functions . . . . .                                               | 39        |
| 9.4       | Exercises . . . . .                                                    | 39        |
| <b>10</b> | <b>Projected Gradient</b>                                              | <b>42</b> |
| 10.1      | Convergence Properties to Stationary Points . . . . .                  | 42        |
| 10.2      | Projected Gradient and Line Search . . . . .                           | 42        |
| 10.3      | Projection Functions . . . . .                                         | 43        |
| 10.3.1    | $n$ -dimensional Sphere . . . . .                                      | 43        |
| 10.3.2    | $n$ -dimensional Box . . . . .                                         | 43        |
| 10.4      | Exercises . . . . .                                                    | 43        |
| <b>11</b> | <b>Quadratic Programming with Eq. Constraints (Lab. 11)</b>            | <b>45</b> |
| 11.1      | Equivalent Definition of $\mathcal{L}$ . . . . .                       | 45        |
| 11.2      | KKT Conditions for QP Problems . . . . .                               | 45        |
| 11.3      | Only Equality Constraints for QP . . . . .                             | 46        |
| 11.4      | Schur Complement Resolution . . . . .                                  | 46        |
| 11.5      | Null Space Method . . . . .                                            | 47        |
| 11.5.1    | Creation of $Z$ . . . . .                                              | 47        |
| 11.5.2    | Creation of $\hat{x}$ . . . . .                                        | 47        |
| 11.6      | Exercises . . . . .                                                    | 48        |
| <b>12</b> | <b>Automatic Differentiation</b>                                       | <b>49</b> |
| 12.1      | Computational Graph . . . . .                                          | 49        |
| 12.2      | Automatic Differentiation: Forward Mode . . . . .                      | 49        |
| 12.3      | Automatic Differentiation: Reverse Mode . . . . .                      | 51        |
| 12.4      | Reverse AD in Matlab . . . . .                                         | 52        |
| 12.5      | Exercises . . . . .                                                    | 52        |
| <b>A</b>  | <b>Material for Refreshing Basic Knowledge</b>                         | <b>53</b> |

|          |                                                            |           |
|----------|------------------------------------------------------------|-----------|
| <b>B</b> | <b>Test Functions - Optimization</b>                       | <b>53</b> |
| B.1      | The Rosenbrock and the Himmelblau Test Functions . . . . . | 53        |
| B.2      | $n$ -dimensional Test Function . . . . .                   | 54        |
| <b>C</b> | <b>Test Functions - Equations and Systems</b>              | <b>56</b> |
| C.1      | Nonlinear Equations . . . . .                              | 57        |

## Introduction

In this document, we report the text of the laboratories for the course of *Numerical Optimization for Large Scale Problems*, A.Y. 2025/2026. Each section corresponds to one laboratory and it will consist of a brief theoretical recap of the arguments, followed by the exercises.

Usually (but not always), the solutions of the exercises of each laboratory are published on the course's web page, before the start of the next one. Solutions will be published only as MATLAB code (except for the first laboratory, published also as Python code).

**Remark 0.1** (Material for refreshing knowledge: MATLAB and basic numerical methods for Linear Algebra). In Section [A](#), we report the content of the message of Prof. Pieraccini on the course web page and related to the available material about “*MATLAB pills*” and “*Background*”. Look at that material to refresh your knowledge about MATLAB and basic numerical methods for Linear Algebra, since they are strongly suggested for the laboratories (and the course in general).

# 1 Direct Methods for Linear Systems

Let us consider a linear system, with coefficients in  $\mathbb{R}$ , of  $m$  equations and  $n$  unknowns

$$A\mathbf{x} = \mathbf{b}, \quad (1)$$

where  $A \in \mathbb{R}^{m \times n}$  denotes the matrix of the *coefficients*,  $\mathbf{x} \in \mathbb{R}^n$  denotes the vectors of the *unknowns*, and  $\mathbf{b} \in \mathbb{R}^m$  is the vector of *known terms*.

Then, we recall that:

1. **Rouché-Capelli's Theorem:** system (1) admits solutions if and only if

$$\text{rank}(A) = \text{rank}(A|\mathbf{b}). \quad (2)$$

In particular, if (2) holds, (1) admits  $\infty^{n-\text{rank}(A)}$  solutions (one solution if  $\text{rank}(A) = n$ ).

2. If  $\mathbf{b} = \mathbf{0}$ , the system (1) is defined as *homogeneous system*. Due to item 1, a homogeneous system always admits solutions (trivial solution:  $\mathbf{x}^* = \mathbf{0}$ ).
3. If  $\mathbf{b} \neq \mathbf{0}$  The set of all and only the solutions of system (1) is

$$\mathcal{S}_{\mathbf{b}} = \{\bar{\mathbf{x}} + \mathbf{x}^* \mid \mathbf{x}^* \in \mathcal{S}_0\}, \quad (3)$$

where  $\bar{\mathbf{x}} \in \mathbb{R}^n$  is a particular solution of (1) (i.e.,  $A\bar{\mathbf{x}} = \mathbf{b}$ ) and  $\mathcal{S}_0 \subseteq \mathbb{R}^n$  is the set of all and only the solutions of the homogeneous system  $A\mathbf{x} = \mathbf{0}$ .

## 1.1 Square Matrices

Here, we focus on the case of linear systems like (1) but where the number of equations is equal to the number of unknowns. Therefore, the matrix of the coefficients is a square matrix  $A \in \mathbb{R}^{n \times n}$  and the vector of known terms is  $\mathbf{b} \in \mathbb{R}^n$  (i.e.,  $m = n$ ).

We recall the following types of square matrices:

- **Diagonal:**  $A$  is diagonal if  $a_{ij} = 0$ , for each  $i \neq j$ .

$$A = \begin{bmatrix} \bullet & & & \\ & \ddots & & \\ & & \ddots & \\ & & & \bullet \end{bmatrix}.$$

- **Lower/Upper Triangular:**  $A$  is lower/upper triangular if  $a_{ij} = 0$ , for each  $i \gtrless j$ .

$$A_{lt} = \begin{bmatrix} \bullet & & \\ \vdots & \ddots & \\ \bullet & \dots & \bullet \end{bmatrix}, \quad A_{ut} = \begin{bmatrix} \bullet & \dots & \bullet \\ & \ddots & \vdots \\ & & \bullet \end{bmatrix}.$$

- **Symmetric:**  $A$  is symmetric if it is equal to its transpose (i.e.,  $A = A^T$ ).
- **Positive/Negative (Semi-)Definite:**  $A$  is positive/negative definite if  $\mathbf{x}^T A \mathbf{x} \gtrless 0$ , for each  $\mathbf{x} \in \mathbb{R}^n \setminus \{\mathbf{0}\}$ .  $A$  is positive/negative semi-definite if  $\mathbf{x}^T A \mathbf{x} \gtrless= 0$ , for each  $\mathbf{x} \in \mathbb{R}^n \setminus \{\mathbf{0}\}$ .
- **Invertible and Inverse Matrix:**  $A$  is *invertible* if exists a matrix  $B \in \mathbb{R}^{n \times n}$  such that  $AB = BA = \mathbb{I}_n$ . Then,  $B$  is defined as *inverse matrix* of  $A$  and is denoted by  $A^{-1}$ .  
A matrix that is not invertible is also called *singular*; then, saying that a matrix is *non-singular* is equivalent to saying that it is *invertible*.
- **Orthogonal:**  $A$  is *orthogonal* if its inverse is equal to its transpose (i.e.,  $A^{-1} = A^T$  and  $AA^T = A^T A = \mathbb{I}_n$ ).

## 1.2 Gaussian Elimination

Here, we briefly recall the *Gaussian Elimination* properties for matrices and linear systems. First of all, we recall that the gaussian elimination operations are:

- **Switch:** switch row  $i$  and row  $j$  of a matrix;
- **Multiplication by Scalar:** multiply the  $i$ -th row of a matrix by a scalar value  $\lambda \in \mathbb{R}$ ;
- **Add Row to Another:** add row  $i$ , multiplied by a scalar  $\lambda \in \mathbb{R}$ , to row  $j$  of a matrix.

We recall that the gaussian elimination operations have the following properties:

- They are invertible linear applications; i.e., their action can be described by left-multiplying  $A$  by a non-singular matrix  $E \in \mathbb{R}^{m \times m}$ .
- Let  $E_1, \dots, E_k$  be the ordered sequence of  $k \in \mathbb{N}$  gaussian elimination operations applied to  $A$  and let  $E \in \mathbb{R}^{m \times m}$  be  $E := E_k \cdots E_1$ . Then:
  - $E$  is non-singular;
  - The linear system  $A'x = b'$ , with  $A' = EA$  and  $b' = Eb$ , has the same solutions of (1).

## 1.3 Square Linear Systems

Concerning square linear systems, we recall that:

1.  **$A$  invertible (i.e., non-singular):** due to Rouché-Capelli's Theorem (see item 1 above), if  $\det(A) \neq 0$ , then (1) admits one unique solution. Moreover, since  $A$  is invertible if and only if  $\det(A) \neq 0$ , the unique solution of (1) is  $x^* = A^{-1}b$ .
2.  **$A$  not invertible (i.e., singular):** if  $A$  is singular, the solutions of the linear system depends on the Rouché-Capelli's Theorem.
3. **Forward/Backward Substitution:** if  $A$  is lower/upper triangular and non-singular, the unique solution  $x^*$  can be easily computed using *forward/backward substitution*. We recall that all the diagonal elements of a non-singular triangular matrix are non-null (i.e.,  $a_{ii} \neq 0$ , for each  $i = 1, \dots, n$ ).

- **Forward Substitution:**

$$x_1^* = \frac{b_1}{a_{11}} \rightarrow x_2^* = \frac{b_2 - a_{21}x_1^*}{a_{22}} \rightarrow \dots \rightarrow x_n^* = \frac{b_n - a_{n(n-1)}x_{(n-1)}^* - \dots - a_{n1}x_1^*}{a_{nn}}.$$

- **Backward Substitution:**

$$x_n^* = \frac{b_n}{a_{nn}} \rightarrow x_{(n-1)}^* = \frac{b_{(n-1)} - a_{(n-1)n}x_n^*}{a_{(n-1)(n-1)}} \rightarrow \dots \rightarrow x_1^* = \frac{b_1 - a_{12}x_2^* - \dots - a_{1n}x_n^*}{a_{11}}.$$

4. **Substitution (Diagonal Special Case):** if  $A$  is non-singular and diagonal, the unique solution of (1) is  $x^* = (b_1/a_{11}, \dots, b_n/a_{nn})$ .
5. **Gaussian Elimination:** if  $A$  is non-singular we can perform a gaussian elimination  $E \in \mathbb{R}^{n \times n}$  such that  $EA$  is a triangular matrix (usually, upper triangular); then, we solve the triangular linear system  $(EA)x = Eb$  by substitution.
6. **LU Factorization:** let  $A$  be non-singular and  $E$  the matrix of gaussian elimination such that  $EA = U$  is upper triangular and  $E^{-1}$  is lower triangular; then,  $LU = E^{-1}EA = A$  and we can solve the linear system (1) by two substitutions:  $y^* = L^{-1}b$ ,  $x^* = U^{-1}y^*$ .

7. **Cholesky Factorization:** if  $A$  is symmetric and positive definite, there is a unique upper triangular matrix  $R$ , with positive diagonal elements, such that the LU factorization of  $A$  is  $A = R^T R$ .
8. **QR Factorization:** with proper operations (other than gaussian elimination),  $A$  can be decomposed as  $A = QR$ , where  $Q$  is orthogonal and  $R$  is upper triangular.

**Observation:** factorizations typically are more convenient than the gaussian elimination if you have to solve many linear systems with the same matrix (e.g.,  $A\mathbf{x} = \mathbf{b}_1, \dots, A\mathbf{x} = \mathbf{b}_N$ ).

**Observation:** the advantages of the QR factorization are:

- methods for obtaining the QR factorization are usually *more stable* (see Section 1.4 below), but more expensive, than the ones used for obtaining the LU factorization;
- QR factorization can be easily extended to *rectangular* linear systems.

## 1.4 Condition Number

The reliability of a solution  $\mathbf{x}^*$  of system (1) depends on the condition number of the matrix  $A$ . We recall that the condition number of  $A$  is

$$k(A) = \|A\| \|A^{-1}\|, \quad (4)$$

where  $\|\cdot\|$  denotes a matrix norm (typically, it is used  $\|\cdot\|_2$ ).

Specifically, let  $A \in \mathbb{R}^{n \times n}$  be a non-singular matrix and let  $\mathbf{x}, \mathbf{b} \in \mathbb{R}^n$  be two vectors such that  $A\mathbf{x} = \mathbf{b}$ . Moreover, let  $\hat{\mathbf{x}} := \mathbf{x} + \delta\mathbf{x}$  and  $\hat{\mathbf{b}} := \mathbf{b} + \delta\mathbf{b}$ , with  $\delta\mathbf{x} \neq \mathbf{0}$  and  $\delta\mathbf{b}$  such that  $A\hat{\mathbf{x}} = \hat{\mathbf{b}}$ .

Then

$$\frac{\|\delta\mathbf{x}\|}{\|\mathbf{x}\|} \leq k(A) \frac{\|\delta\mathbf{b}\|}{\|\mathbf{b}\|}, \quad (5)$$

where here  $\|\cdot\|$  denotes a vector norm (typically, it is used  $\|\cdot\|_2$ ).

**Observation:**  $k(A) \geq \|AA^{-1}\| = \|\mathbb{I}_n\| = 1$ ; i.e.,  $k(A) \in [1, +\infty)$ .

**Observation:** the smaller  $k(A)$ , the more reliable the solution  $\mathbf{x}^*$  of system (1). On the contrary, the larger  $k(A)$ , the larger the upper bound for the noise of the solution and, by consequence, the lower the reliability of  $\mathbf{x}^*$ .

If a matrix  $A$  is characterized by a large condition number (i.e.,  $k(A) \gg 1$ ), the matrix is *ill-conditioned*.

## 1.5 Direct Methods in Matlab

In Figures 1 and 2, the schemes of the Matlab procedure for solving linear systems are reported. In particular, we have two different schemes depending on the method used to store matrices: *dense* matrices and *sparse* matrices. In general, for solving a linear system in Matlab using a direct method, the `mldivide` function is used (equivalent to the operator `\`). [Here](#), the online documentation.

## 1.6 Direct Methods in Python

For solving linear systems in Python, the `numpy` and `scipy` modules are strongly suggested; indeed, they have many built-in linear algebra tools. In general, the sub-modules `numpy.linalg` and `scipy.linalg` are almost equivalent; on the other hand, for working with sparse matrices, the sub-modules `scipy.sparse` and `scipy.sparse.linalg` are necessary. In particular, for solving linear systems you can use `numpy.linalg.solve` or `scipy.linalg.solve` for dense matrices and `scipy.sparse.linalg.spsolve` for sparse matrices. The schemes adopted by these procedures are similar to the ones of `mldivide` in Matlab.

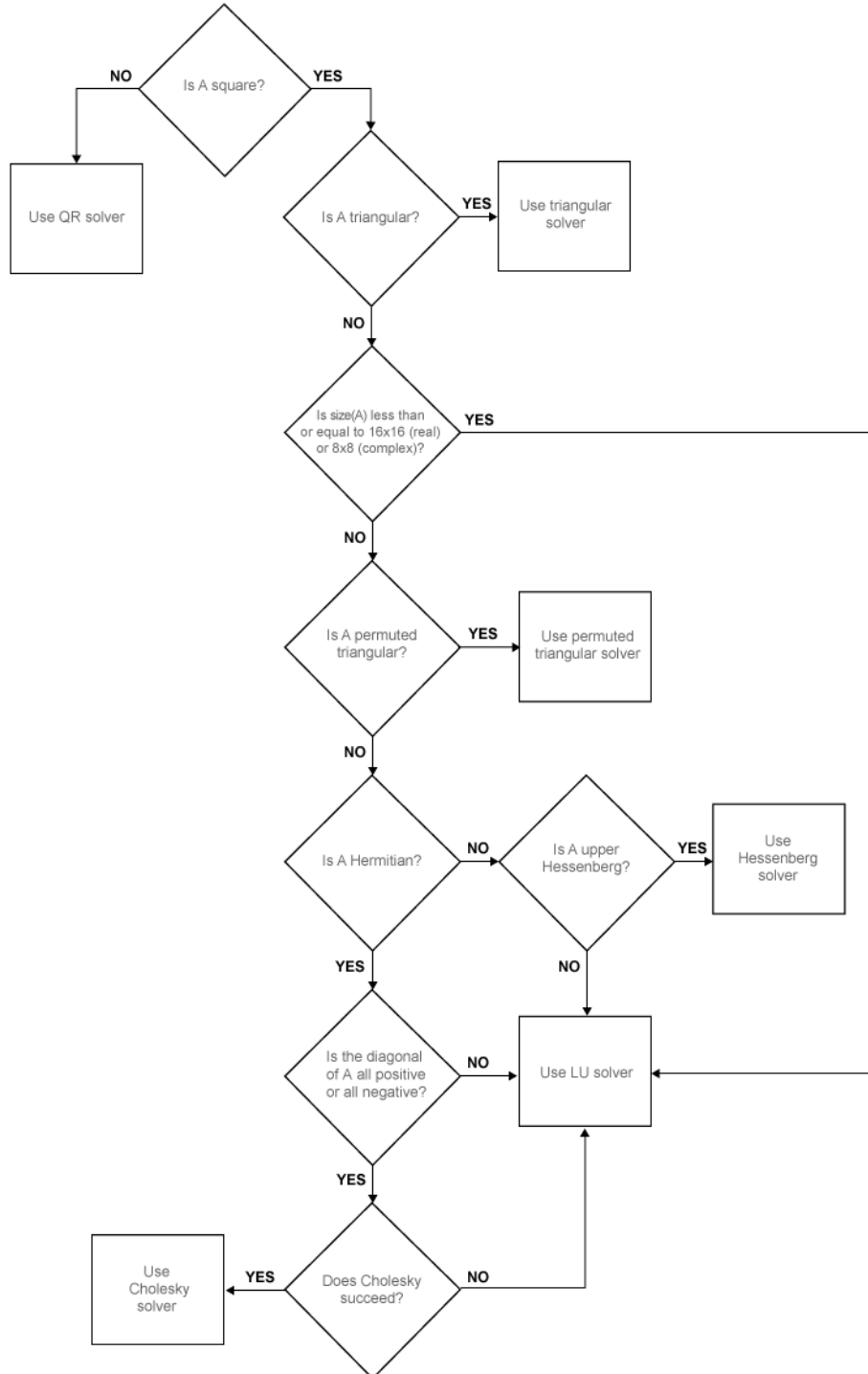


Figure 1: Scheme for the `mldivide` (`\`) function in Matlab (dense matrices case).



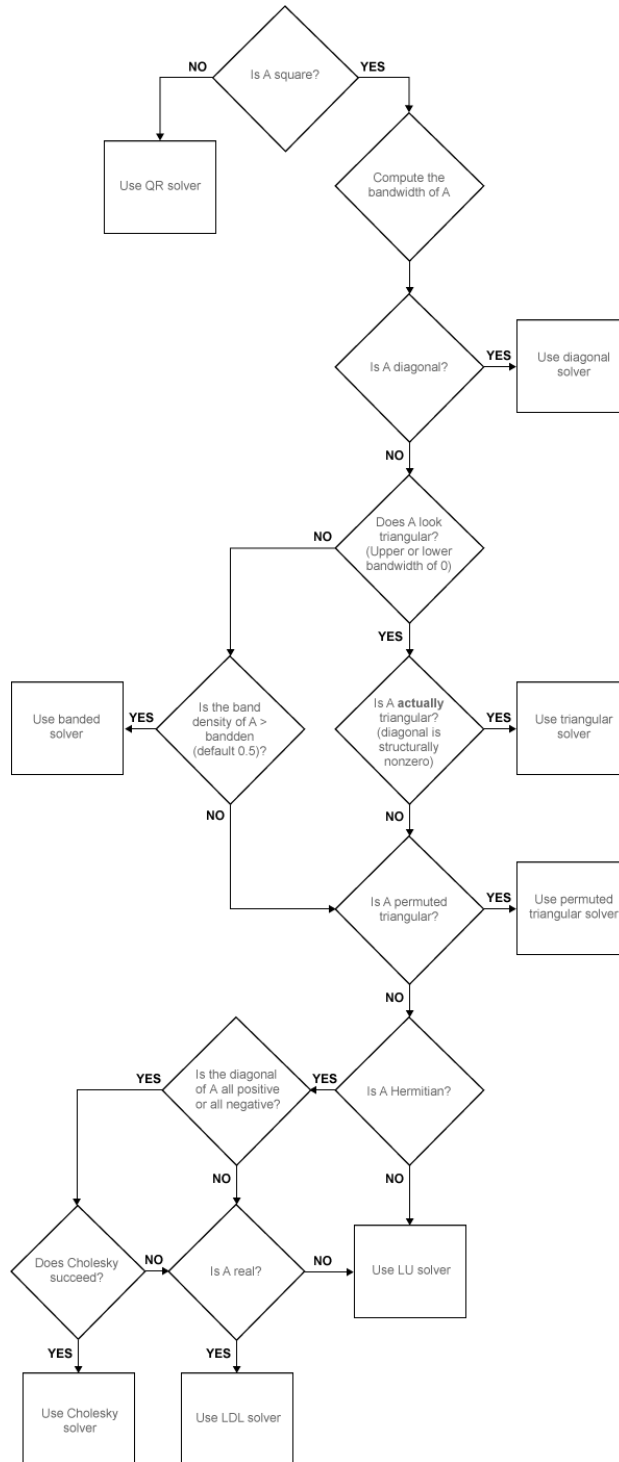


Figure 2: Scheme for the `mldivide` (`\`) function in Matlab (sparse matrices case).

## 1.7 Fill-In Phenomenon

We recall that a matrix is *sparse* if most of its elements are equal to zero. Sparse matrices have specific storage methods for reducing memory storage without losing efficiency in matrix operations.

Nonetheless, direct methods are not suitable for very large sparse matrices, due to the *fill-in* phenomenon. In particular, we have that a factorization of a sparse matrix  $A$  can return two *dense* matrices. This phenomenon can lead to memory storage problems and/or expensive computations of the factors.

## 1.8 Exercises

**Exercise 1.1** (Basic Linear Systems). Write a script file where:

1. The random seed is 1.
2. A matrix  $A \in \mathbb{R}^{n \times n}$  is initialized with elements randomly sampled with uniform distribution in  $[0, 1]$ . The value of  $n$  can be initialized as  $n = 5$  for simplicity.
3. A vector  $\mathbf{b} \in \mathbb{R}^n$  is initialized such that the exact solution of  $A\mathbf{x} = \mathbf{b}$  is the vector  $\mathbf{x}^* = \mathbf{e} = (1, \dots, 1) \in \mathbb{R}^n$ .
4. A set of  $N$  vectors  $\mathbf{b}_1, \dots, \mathbf{b}_N \in \mathbb{R}^n$  are initialized such that  $\mathbf{b}_i = \mathbf{b} + \boldsymbol{\xi}_i$ ,  $\boldsymbol{\xi}_i \in \mathcal{U}([0, 1]^n)$ , for each  $i = 1, \dots, N$ . The value of  $N$  can be initialized as  $N = 10$  for simplicity.
5. A diagonal matrix  $D \in \mathbb{R}^{n \times n}$  is initialized with elements randomly sampled with uniform distribution in  $[0, 1]$ .
6. Solve the linear systems  $A\mathbf{x} = \mathbf{b}$ ,  $D\mathbf{x} = \mathbf{b}$ , and  $A\mathbf{x} = \mathbf{b}_i$ , for each  $i = 1, \dots, N$ .
7. Compute the norms of the residuals of the numerical solutions; i.e.,  $\|A\hat{\mathbf{x}} - \mathbf{b}\|$ ,  $\|D\hat{\mathbf{x}} - \mathbf{b}\|$ , and  $\|A\hat{\mathbf{x}} - \mathbf{b}_i\|$ , for each  $i = 1, \dots, N$ .
8. Compute the condition number of  $A$  and  $D$  and comment the results.

**Exercise 1.2** (Ill-Conditioned Problem). Write a script file where you solve a linear system  $A\mathbf{x} = \mathbf{b}$  such that

$$A = \begin{bmatrix} 0.835 & 0.667 \\ 0.333 & 0.266 \end{bmatrix},$$

and  $\mathbf{b} \in \mathbb{R}^2$  is the result of an experiment, measured using an instrument with tolerance  $\pm 0.001$ . For simplicity, assume to have the following 11 measurements of  $\mathbf{b}$ :

$$\mathbf{b}_i = \mathbf{b}^* - 0.001 + (i - 1)0.0002, \quad \forall i = 1, \dots, 11,$$

where  $\mathbf{b}^* = (0.168, 0.067)$  is the exact vector of known terms.

Then:

1. Solve the linear systems  $A\mathbf{x} = \mathbf{b}_i$ , for each  $i = 1, \dots, N$ .
2. Compute the norms of the residuals of the numerical solutions; i.e.,  $\|A\hat{\mathbf{x}} - \mathbf{b}_i\|$ , for each  $i = 1, \dots, N$ .
3. Compute the condition number of  $A$  and comment the results.

**Exercise 1.3** (Problem Conditioning). Write a function that takes as inputs a random seed, the dimension  $n$  and the number of samples  $N$ , and that runs the same operations of the script of Exercise 1.1, returning:

1.  $A, \mathbf{b}$ , and  $D$ ;
2. the solutions of  $A\mathbf{x} = \mathbf{b}$ ,  $D\mathbf{x} = \mathbf{b}$ , and  $A\mathbf{x} = \mathbf{b}_i$ , for each  $i = 1, \dots, N$ ;
3.  $k(A)$  and  $k(D)$ .

Try the function for different random seeds (e.g., 42) and try to understand if the linear systems' solutions are reliable or not. Then, try also to change dimension  $n$  and number of samples  $N$ .

**Exercise 1.4.** Download, run, and read the script `lab01_sparse_examples.m/.py` carefully. Then write a new script file where, for each  $n = 200, 400, \dots, 2000$ , the following operations are performed:

- Initialize a random sparse matrix  $A \in \mathbb{R}^{n \times n}$  with density 0.01 (0.1 in Python);
- Compute the exact density of non-zero elements of  $A$ ;
- Compute  $L, U \in \mathbb{R}^{n \times n}$  through the LU Factorization of  $A$ ;
- Compute the exact density of non-zero elements of  $L$  and  $U$ ;
- Update the script file, increasing the maximum value of  $n$  and looking for the value that run out of memory your PC.

In the end, plot the evolution of the density values for  $A$ ,  $L$ , and  $U$ , with respect to  $n$  and comment the results. Moreover, spy the collocation of the non-zero values of the last matrices  $A$ ,  $L$ , and  $U$  computed.

**Suggestion:** write a function that takes a random seed, the dimension  $n$ , and the desired density of  $A$  as inputs, and that returns all you need to perform the operations asked by the exercise.

**Exercise 1.5.** Write a new script that performs the same operations of Exercise 1.4 but using only dense matrices. Observe when your PC runs out of memory.

**Exercise 1.6.** Write a new script that performs the same operations of Exercise 1.4 but using the incomplete LU factorization of the matrix  $\tilde{A} = A + \mathbb{I}$ . Measure and spy the density of the factorization matrices  $\tilde{L}, \tilde{U}$ , and the compute error norm  $\| \tilde{L}\tilde{U} - \tilde{A}P \|$  and the relative error norm  $\| \tilde{L}\tilde{U} - \tilde{A}P \| / \| A \|$ .

## 2 Finite Differences

We briefly recall the main Finite Difference formulas for approximating derivatives, to help carry out the exercises of this section.

**Notation 2.1** (Notations for Gradient, Hessian, and Jacobian). *Let  $f : \Omega \subseteq \mathbb{R}^n \rightarrow \mathbb{R}$ ,  $\mathbf{F} = (f_1, \dots, f_m) : \Omega \subseteq \mathbb{R}^n \rightarrow \mathbb{R}^m$ , then:*

- Gradient of  $f$  (differentiable in  $\Omega$ ):

$$\nabla f = \begin{bmatrix} f_{x_1} \\ \vdots \\ f_{x_n} \end{bmatrix} : \mathbb{R}^n \rightarrow \mathbb{R}^n, \quad (6)$$

where  $f_{x_i} := \partial f / \partial x_i$ , for each  $i = 1, \dots, n$ .

- Hessian of  $f$  (twice differentiable in  $\Omega$ ):

$$Hf = \nabla^2 f = \left( \frac{\partial^2}{\partial x_j \partial x_i} f \right)_{ij} = \begin{bmatrix} f_{x_1 x_1} & \cdots & f_{x_1 x_n} \\ \vdots & \ddots & \vdots \\ f_{x_n x_1} & \cdots & f_{x_n x_n} \end{bmatrix} : \mathbb{R}^n \rightarrow \mathbb{R}^{n \times n}, \quad (7)$$

where  $f_{x_i x_j} := \partial^2 f / (\partial x_j \partial x_i)$ , for each  $i, j = 1, \dots, n$ .

**Attention:** if  $f \in C^2(\Omega)$ ,  $Hf(\mathbf{x})$  is symmetric for each  $\mathbf{x} \in \Omega$ . Otherwise, symmetry is not guaranteed!

- Jacobian of  $\mathbf{F}$  ( $f_1, \dots, f_m$  differentiable in  $\Omega$ ):

$$J\mathbf{F} = \left( \frac{\partial}{\partial x_j} f_i \right)_{ij} = \begin{bmatrix} \frac{\partial}{\partial x_1} f_1 & \cdots & \frac{\partial}{\partial x_n} f_1 \\ \vdots & \ddots & \vdots \\ \frac{\partial}{\partial x_1} f_m & \cdots & \frac{\partial}{\partial x_n} f_m \end{bmatrix} = \begin{bmatrix} \nabla f_1^T \\ \vdots \\ \nabla f_m^T \end{bmatrix} : \mathbb{R}^n \rightarrow \mathbb{R}^{m \times n}. \quad (8)$$

### 2.1 Finite Differences (Gradient)

If  $f : \Omega \subseteq \mathbb{R}^n \rightarrow \mathbb{R}$  is  $C^1(\Omega)$ , we can approximate the gradient of  $f$  in  $\hat{\mathbf{x}} \in \Omega$  with the following expressions.

- Forward Finite Differences (FW-FD):

$$f_{x_i}(\hat{\mathbf{x}}) = \frac{\partial}{\partial x_i} f(\hat{\mathbf{x}}) \approx \frac{f(\hat{\mathbf{x}} + h\mathbf{e}_i) - f(\hat{\mathbf{x}})}{h}, \quad \forall i = 1, \dots, n; \quad (9)$$

- Centered Finite Differences (C-FD):

$$f_{x_i}(\hat{\mathbf{x}}) = \frac{\partial}{\partial x_i} f(\hat{\mathbf{x}}) \approx \frac{f(\hat{\mathbf{x}} + h\mathbf{e}_i) - f(\hat{\mathbf{x}} - h\mathbf{e}_i)}{2h}, \quad \forall i = 1, \dots, n. \quad (10)$$

#### 2.1.1 Choice of $h$

Large values for  $h$  return obviously poor approximations, but too small values are characterized by numerical cancellation problems.

In FW-FD, a suggested trade-off is to select  $h = \sqrt{\varepsilon_m}$  (see [2] for motivations), or  $h_i = \sqrt{\varepsilon_m} |\hat{x}_i|$  for approximating the partial derivative  $f_{x_i}$ . For simplicity, this value can be adopted also for C-FD, even if better values of  $h$  exist.

## 2.2 Finite Differences (Hessian)

If the function  $f : \Omega \subseteq \mathbb{R}^n \rightarrow \mathbb{R}$  is  $C^2(\Omega)$  (i.e., symmetric Hessian for each  $\mathbf{x} \in \Omega$ ), we can approximate the second order derivatives of  $f$  in  $\hat{\mathbf{x}} \in \Omega$  with the following expressions.

- *Hessian's diagonal elements:*

$$(Hf)_{ii}(\hat{\mathbf{x}}) = f_{x_i x_i}(\hat{\mathbf{x}}) \approx \frac{f(\hat{\mathbf{x}} + h\mathbf{e}_i) - 2f(\hat{\mathbf{x}}) + f(\hat{\mathbf{x}} - h\mathbf{e}_i)}{h^2}, \quad (11)$$

for each  $i = 1, \dots, n$ ;

- *Hessian's non-diagonal elements:*

$$(Hf)_{ij}(\hat{\mathbf{x}}) = f_{x_i x_j}(\hat{\mathbf{x}}) \approx \frac{f(\hat{\mathbf{x}} + h\mathbf{e}_i + h\mathbf{e}_j) - f(\hat{\mathbf{x}} + h\mathbf{e}_i) - f(\hat{\mathbf{x}} + h\mathbf{e}_j) + f(\hat{\mathbf{x}})}{h^2}, \quad (12)$$

for each  $i, j = 1, \dots, n, i \neq j$ .

### 2.2.1 Choice of $h$

The choice of  $h$  for the Hessian, in order to avoid numerical cancellation problems, is suggested to be greater than the  $h$  used for the gradient, due to the  $h^2$  in the denominator. Then, for example, we can use an  $h$  that is the square root of the value that we would use for the gradient:  $h = \sqrt{h_{grad}}$ .

## 2.3 Finite Differences (Jacobian)

If  $\mathbf{F} = (f_1, \dots, f_m) : \Omega \subseteq \mathbb{R}^n \rightarrow \mathbb{R}^m$  is such that  $f_1, \dots, f_m \in C^1(\Omega)$ , we can approximate the Jacobian of  $\mathbf{F}$  in  $\hat{\mathbf{x}} \in \Omega$  with the following expressions.

- *Forward Finite Differences ( $j$ -th Jacobian's column):*

$$(J\mathbf{F})_{\cdot j}(\hat{\mathbf{x}}) \approx \frac{\mathbf{F}(\hat{\mathbf{x}} + h\mathbf{e}_j) - \mathbf{F}(\hat{\mathbf{x}})}{h}, \quad \forall j = 1, \dots, n; \quad (13)$$

- *Centered Finite Differences ( $j$ -th Jacobian's column):*

$$(J\mathbf{F})_{\cdot j}(\hat{\mathbf{x}}) \approx \frac{\mathbf{F}(\hat{\mathbf{x}} + h\mathbf{e}_j) - \mathbf{F}(\hat{\mathbf{x}} - h\mathbf{e}_j)}{2h}, \quad \forall j = 1, \dots, n; \quad (14)$$

**Attention:** remember that is possible to approximate more columns at the same times if  $J\mathbf{F}$  is sparse (see [2]).

## 2.4 Exercises

The text of the exercises is for Matlab users. Nonetheless, you can use Python instead.

**Exercise 2.1** (Finite Differences - Gradient). *Write a Matlab function finddiff\_grad.m that implements both the forward and centered finite difference methods for the gradient computation, given:*

- $f$ : a function handle variable that, for each column vector  $\mathbf{x} \in \mathbb{R}^n$ , returns  $f(\mathbf{x})$ , where  $f : \Omega \subseteq \mathbb{R}^n \rightarrow \mathbb{R}$  is the function whose gradient we want to calculate;
- $\mathbf{x}$ : a column vector of  $n$  elements representing the point where we want to compute the gradient of  $f$ ;
- $h$ : the finite difference “step”;

- **type**: a string of characters such that if it is 'fw', the function uses the forward method (default), and if it is 'c', the function uses the centered method.

The output (i.e., the gradient approximation) must be stored as a column vector.

Once you have written the function, test it using the functions in Section B.1 and compare the results with the exact gradients.

**Exercise 2.2** (Finite Differences - Jacobian). Write a Matlab function `findiff_J.m` that implements both the forward and centered finite difference methods for the Jacobian computation, given:

- **F**: a function handle variable that, for each column vector  $\mathbf{x} \in \mathbb{R}^n$ , returns  $\mathbf{F}(\mathbf{x})$ , where  $\mathbf{F} : \Omega \subseteq \mathbb{R}^n \rightarrow \mathbb{R}^m$  is the function whose Jacobian we want to calculate;
- **x**: a column vector of  $n$  elements representing the point where we want to compute the Jacobian of  $\mathbf{F}$ ;
- **h**: the finite difference “step”;
- **type**: a string of characters such that if it is 'fw', the function uses the forward method (default), and if it is 'c', the function uses the centered method.

Once you have written the function, test it using the functions in Section B.1 for building a vectorial test function  $\mathbf{F}$  and compare the results with the exact gradients.

**Exercise 2.3** (Finite Differences - Hessian). Write a Matlab function `findiff_Hess.m` that implements the finite difference method for the Hessian computation (symmetric case only), given:

- **f**: a function handle variable that, for each column vector  $\mathbf{x} \in \mathbb{R}^n$ , returns  $f(\mathbf{x})$ , where  $f : \Omega \subseteq \mathbb{R}^n \rightarrow \mathbb{R}$  is the  $C^2$  function whose Hessian we want to calculate;
- **x**: a column vector of  $n$  elements representing the point where we want to compute the gradient of  $f$ ;
- **h**: the finite difference “step”.

Once you have written the function, test it using the functions in Section B.1 and compare the results with the exact Hessians.

**Exercise 2.4** (Finite Differences - Hessian as Jacobian of the Gradient). Modify the Matlab function `findiff_J.m` such that it can force the symmetry in the output matrix.

Once you have written the function, test it using the functions in Section B.1 for computing the Hessians, given the exact gradients; then, compare the results with the exact Hessians.

**Exercise 2.5** (Finite Differences - Approximation Quality and  $h$ ). Write a Matlab script `lab-02_FD_tests.m` where you compute the gradient, Jacobian, and Hessian of some test functions, varying the value of  $h$ . Analyze and comment on the results.

**Exercise 2.6** (Finite Differences - Smart Implementations). Look at the test function illustrated in B.2. Then, write a Matlab custom functions for approximating efficiently with Finite Differences the gradient and the Hessian (both directly or as Jacobian of the exact gradient).

**Suggestion:** exploit the summation structure for the gradient and the sparsity for the Hessian.

### 3 Iterative Methods for Linear Systems

As we stated at the end of the Section 1, direct methods are not particularly suitable for linear systems with very large sparse matrix, due to the *fill-in* phenomenon (see Section 1.7). On the contrary, *iterative methods* can preserve the advantages given by a sparse matrix.

The main idea behind an iterative method is the following procedure:

- Choose a *starting guess*  $\mathbf{x}^{(0)}$  for the solution of the problem;
- Build a sequence  $\{\mathbf{x}^{(k)}\}_{k \in \mathbb{N}}$  such that:
  - $\mathbf{x}^{(k+1)}$  is computed using information of step  $k$  (or even previous steps);
  - $\mathbf{x}^{(k)} \xrightarrow{k \rightarrow \infty} \mathbf{x}^*$ , where  $\mathbf{x}^*$  is an exact solution of the problem.

Since the sequence of vectors  $\{\mathbf{x}^{(k)}\}_{k \in \mathbb{N}}$  is infinite, the introduction of stopping criteria is necessary for returning an approximated solution  $\hat{\mathbf{x}} \approx \mathbf{x}^*$ . Concerning the methods for linear systems there can be:

- **Relative Stopping Criteria:**

- *Relative Increment*:  $\|\mathbf{x}^{(k+1)} - \mathbf{x}^{(k)}\| / \|\mathbf{x}^{(k+1)}\| < \tau$ ;
- *Relative Residual*:  $\|\mathbf{r}^{(k+1)}\| / \|\mathbf{b}\| < \tau$ , where  $\mathbf{r}^{(k)} := \mathbf{b} - \mathbf{A}\mathbf{x}^{(k)}$  is the *residual* for the “guess”  $\mathbf{x}^{(k)}$ ,  $k \in \mathbb{N}$ ;
- ...

- **Absolute Stopping Criteria:**

- *Absolute Increment*:  $\|\mathbf{x}^{(k+1)} - \mathbf{x}^{(k)}\| < \tau$ ;
- *Absolute Residual*:  $\|\mathbf{r}^{(k+1)}\| < \tau$ ;
- ...

**Tolerance and Stopping Criteria:** the value  $\tau$  used above for describing the stopping criteria is known as *tolerance* and it is a positive real value ( $\tau > 0$ ), usually very small (i.e.,  $\tau \approx 0$ ). Indeed, the smaller the relative/absolute quantities at step  $k + 1$ , the more we can *assume*  $\mathbf{x}^{(k+1)} \approx \mathbf{x}^*$ .

In the end of this introduction, concerning the iterative methods for linear systems we can observe that:

- There is not a best stopping criterion. The best choice depends on the problem characteristics;
- The smaller tolerance  $\tau$ , the larger the number of iterations required. On the other hand, the smaller the tolerance, the higher the accuracy in approximating  $\mathbf{x}^*$ ;
- Ill-conditioning has effects also on iterative methods. Indeed, at step  $k$ , the inequality (5) can be translated into:

$$\frac{\|\mathbf{x}^{(k)} - \mathbf{x}^*\|}{\|\mathbf{x}^{(k)}\|} \leq k(A) \frac{\|\mathbf{r}^{(k)}\|}{\|\mathbf{b}\|}. \quad (15)$$

Then, even if we use a relative residual stopping criteria with  $\tau$  very small ( $\|\mathbf{r}^{(k)}\| / \|\mathbf{b}\|$ ), we only know that  $\|\mathbf{x}^{(k)} - \mathbf{x}^*\| / \|\mathbf{x}^{(k)}\| \leq k(A)\tau$ .

### 3.1 The Gradient Method

Let us consider a square linear system like (1), where  $A$  is *symmetric* and positive definite (see Section 1.1); necessarily,  $A$  is non-singular. Let  $J : \mathbb{R}^n \rightarrow \mathbb{R}$  be a quadratic function such that

$$J(\mathbf{x}) = \frac{1}{2} \mathbf{x}^T A \mathbf{x} - \mathbf{x}^T \mathbf{b}. \quad (16)$$

Therefore,  $J$  is a (quadratic) strictly convex function defined on  $\mathbb{R}^n$  and it has a unique (global) minimum in  $\mathbf{x}^*$ . Now we recall the following theorems.

**Theorem 3.1** (First Order Necessary Conditions [1]). *Let  $\mathbf{x}^* \in \mathbb{R}^n$  and let  $f : \mathbb{R}^n \rightarrow \mathbb{R}$  be such that it is continuously differentiable in an open neighborhood of  $\mathbf{x}^*$ . If  $\mathbf{x}^*$  is a local minimizer of  $f$ , then  $\nabla f(\mathbf{x}^*) = \mathbf{0}$ .*

**Theorem 3.2** (Second Order Necessary Conditions [1]). *Let  $\mathbf{x}^* \in \mathbb{R}^n$  and let  $f : \mathbb{R}^n \rightarrow \mathbb{R}$  be such that its Hessian  $Hf : \mathbb{R}^n \rightarrow \mathbb{R}^{n \times n}$  exists in  $\mathbf{x}^*$  and is continuous in an open neighborhood of  $\mathbf{x}^*$ . If  $\mathbf{x}^*$  is a local minimizer of  $f$ , then  $\nabla f(\mathbf{x}^*) = \mathbf{0}$  and  $Hf(\mathbf{x}^*)$  is positive semi-definite.*

**Theorem 3.3** (Second Order Sufficient Conditions [1]). *Let  $\mathbf{x}^* \in \mathbb{R}^n$  and let  $f : \mathbb{R}^n \rightarrow \mathbb{R}$  be such that its Hessian  $Hf$  exists in  $\mathbf{x}^*$  and is continuous in an open neighborhood of  $\mathbf{x}^*$ . If  $\nabla f(\mathbf{x}^*) = \mathbf{0}$  and  $Hf(\mathbf{x}^*)$  is positive definite, then  $\mathbf{x}^*$  is a strict local minimizer of  $f$ .*

Now, we observe that  $\nabla J(\mathbf{x}) = A\mathbf{x} - \mathbf{b}$  and  $H_J(\mathbf{x}) = A$ , for each  $\mathbf{x} \in \mathbb{R}^n$ . Therefore, we have that the global minimizer  $\mathbf{x}^*$  of  $J$  is also the solution of the linear system  $A\mathbf{x} = \mathbf{b}$ , indeed:

- $\mathbf{x}^*$  minimizer of  $J \Rightarrow \nabla J(\mathbf{x}^*) = A\mathbf{x}^* - \mathbf{b} = \mathbf{0}$  (Theorem 3.1);
- $\mathbf{x}^*$  solution of  $A\mathbf{x} = \mathbf{b}$  and  $A$  symmetric positive definite  $\Rightarrow \mathbf{x}^*$  (strict) minimizer of  $J$  (Theorem 3.3);
- $A$  is non-singular  $\Rightarrow A\mathbf{x} = \mathbf{b}$  has a unique solution;
- $J$  is strictly convex  $\Rightarrow$  has a unique minimizer.

**Main Idea:** for finding the solution of the linear system, we find instead the minimizer of  $J$  with an iterative method, instead of using a direct method on the linear system. For doing so, we recall the definition of descent direction (see Definition 3.1 below).

**Definition 3.1** (Descent Direction). *A vector  $\mathbf{p} \in \mathbb{R}^n$  is a descent direction for a function  $f : \mathbb{R}^n \rightarrow \mathbb{R}$  at  $\mathbf{x} \in \mathbb{R}^n$  if exists  $\varepsilon > 0$  such that*

$$f(\mathbf{x} + \alpha \mathbf{p}) < f(\mathbf{x}), \quad (17)$$

for each  $0 < \alpha \leq \varepsilon$ . Moreover, let  $f$  be continuously differentiable at  $\mathbf{x}$ ; then,  $\mathbf{p}$  is a descent direction for  $f$  at  $\mathbf{x}$  if and only if

$$\nabla f(\mathbf{x})^T \mathbf{p} < 0. \quad (18)$$

See Figure 3 for an example in  $\mathbb{R}^2$ .

**Remark 3.1** (Steepest Descent/Ascent Direction). Given Definition 3.1, we observe that  $\nabla f(\mathbf{x})$  and  $-\nabla f(\mathbf{x})$  are the directions of steepest ascent and descent, respectively, for  $f$  in  $\mathbf{x}$ .



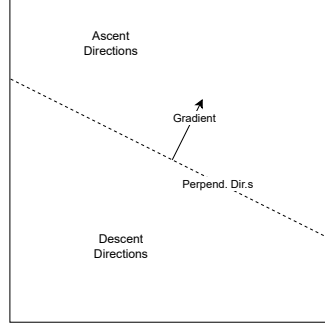


Figure 3: Example of regions of ascent, perpendicular, and descent directions in  $\mathbb{R}^2$ , with respect to a given gradient vector.

### 3.1.1 Sequence of the Gradient Method

Gathering all the knowledge described above, the iterative *gradient method* used for solving the linear system (i.e., minimizing  $J$ ) consists in building the following sequence:

$$\begin{cases} \mathbf{x}^{(0)} \in \mathbb{R}^n, \\ \mathbf{x}^{(k+1)} = \mathbf{x}^{(k)} - \alpha^{(k)} \nabla J(\mathbf{x}^{(k)}), \end{cases} \quad \begin{matrix} \text{given} \\ \forall k \geq 0 \end{matrix}, \quad (19)$$

where  $\alpha^{(k)} \in \mathbb{R}_+$  is the best step length for reaching the minimum available value for  $J(\mathbf{x}^{(k+1)})$ .

In particular, denoting by  $\mathbf{r}^{(k)}$  the  $k$ -th residual, i.e.

$$\mathbf{r}^{(k)} := \mathbf{b} - A\mathbf{x}^{(k)}, \quad \forall k \geq 0, \quad (20)$$

we have that  $\mathbf{r}^{(k)} \equiv -\nabla J(\mathbf{x}^{(k)})$  and, therefore, the sequence can be rewritten as

$$\begin{cases} \mathbf{x}^{(0)} \in \mathbb{R}^n, \\ \mathbf{x}^{(k+1)} = \mathbf{x}^{(k)} + \alpha^{(k)} \mathbf{r}^{(k)}, \end{cases} \quad \begin{matrix} \text{given} \\ \forall k \geq 0 \end{matrix}, \quad (21)$$

where  $\alpha^{(k)} = (\mathbf{r}^{(k)T} \mathbf{r}^{(k)}) / (\mathbf{r}^{(k)T} A \mathbf{r}^{(k)})$ , because  $\frac{d}{d\alpha} J(\mathbf{x} + \alpha \mathbf{r}) = \dots = -\mathbf{r}^T \mathbf{r} + \alpha \mathbf{r}^T A \mathbf{r}$ .

In the following, we report two pseudo-codes for the implementation of the gradient method.

**Algorithm 3.1** (Gradient Method (naive)).

```

1:  $\mathbf{x}^{(0)}$  given
2:  $k \leftarrow 0$ 
3: while stopping criteria are not satisfied do
4:    $\mathbf{r}^{(k)} \leftarrow \mathbf{b} - A\mathbf{x}^{(k)}$ 
5:    $\alpha^{(k)} \leftarrow (\mathbf{r}^{(k)T} \mathbf{r}^{(k)}) / (\mathbf{r}^{(k)T} A \mathbf{r}^{(k)})$ 
6:    $\mathbf{x}^{(k+1)} \leftarrow \mathbf{x}^{(k)} + \alpha^{(k)} \mathbf{r}^{(k)}$ 
7:    $k \leftarrow k + 1$ 
8: end while
9:  $\hat{\mathbf{x}} \leftarrow \mathbf{x}^{(k)}$ 
10: return :  $\hat{\mathbf{x}}$ 

```

**Algorithm 3.2** (Gradient Method).

```

1:  $\mathbf{x}^{(0)}$  given
2:  $\mathbf{r}^{(0)} \leftarrow \mathbf{b} - A\mathbf{x}^{(0)}$ 
3:  $k \leftarrow 0$ 
4: while stopping criteria are not satisfied do

```

```

5:    $\mathbf{z}^{(k)} \leftarrow A\mathbf{r}^{(k)}$ 
6:    $\alpha^{(k)} \leftarrow (\mathbf{r}^{(k)T} \mathbf{r}^{(k)}) / (\mathbf{r}^{(k)T} \mathbf{z}^{(k)})$ 
7:    $\mathbf{x}^{(k+1)} \leftarrow \mathbf{x}^{(k)} + \alpha^{(k)} \mathbf{r}^{(k)}$ 
8:    $\mathbf{r}^{(k+1)} \leftarrow \mathbf{r}^{(k)} - \alpha^{(k)} \mathbf{z}^{(k)}$   $\triangleright$  because  $\mathbf{r}^{(k+1)} = \mathbf{b} - A(\mathbf{x}^{(k)} + \alpha^{(k)} \mathbf{r}^{(k)})$ 
9:    $k \leftarrow k + 1$ 
10: end while
11:  $\hat{\mathbf{x}} \leftarrow \mathbf{x}^{(k)}$ 
12: return :  $\hat{\mathbf{x}}$ 

```

### 3.1.2 Convergence Properties

The convergence of the gradient method is characterized by the following property:

$$\|\mathbf{x}^{(k)} - \mathbf{x}^*\|_A \leq 2 \left( \frac{\kappa_2(A) - 1}{\kappa_2(A) + 1} \right)^k \|\mathbf{x}^{(0)} - \mathbf{x}^*\|_A, \quad \forall k \geq 0, \quad (22)$$

where  $\|\cdot\|_A$  is the *energy norm* such that  $\|\mathbf{x}\|_A := \sqrt{\mathbf{x}^T A \mathbf{x}}$ , and  $\kappa_2(A) := \|A\|_2 \|A^{-1}\|_2 = \lambda_{\max}/\lambda_{\min}$  ( $\lambda$  denotes eigenvalues of  $A$ ).

**Observation:** the more ill-conditioned  $A$ , the larger is the upper bound of  $\|\mathbf{x}^{(k)} - \mathbf{x}^*\|_A$  and, therefore, the slower can be the convergence.

## 3.2 Conjugate Gradient Method

The idea behind the *conjugate gradient* (CG) method is to modify the gradient method using a different descent direction, other than  $\mathbf{r} = -\nabla J(\mathbf{x})$ , for improving the convergence.

Without going into details, the CG method uses as descent directions the sequence

$$\begin{cases} \mathbf{p}^{(0)} = \mathbf{r}^{(0)} \\ \mathbf{p}^{(k+1)} = \mathbf{r}^{(k+1)} + \beta^{(k+1)} \mathbf{p}^{(k)}, \quad \forall k \geq 0 \end{cases}, \quad (23)$$

where  $\beta^{(k+1)} = -(\mathbf{p}^{(k)T} A \mathbf{r}^{(k+1)}) / (\mathbf{p}^{(k)T} A \mathbf{p}^{(k)})$ . Then, the sequence build by the CG for finding  $\mathbf{x}^*$  is

$$\begin{cases} \mathbf{x}^{(0)} \in \mathbb{R}^n, & \text{given} \\ \mathbf{x}^{(k+1)} = \mathbf{x}^{(k)} + \alpha^{(k)} \mathbf{p}^{(k)}, & \forall k \geq 0 \end{cases}, \quad (24)$$

where  $\alpha^{(k)} = (\mathbf{r}^{(k)T} \mathbf{p}^{(k)}) / (\mathbf{p}^{(k)T} A \mathbf{p}^{(k)})$ .

The advantages and disadvantages of CG method are in the following list:

- + *Finite termination:* the CG method reaches the solution  $\mathbf{x}^* \in \mathbb{R}^n$  in at most  $n$  steps;
- if  $n \gg 1$ , the advantages of the finite termination can be lost;
- Due to arithmetic errors, exact conjugacy can be lost;
- ± Even losing the exact conjugacy, we can implement CG as an iterative method with stopping criteria.

In the following, we report two pseudo-codes for the implementation of the CG method.

**Algorithm 3.3** (Conjugate Gradient Method (naive)).

```

1:  $\mathbf{x}^{(0)}$  given
2:  $\mathbf{r}^{(0)} \leftarrow \mathbf{b} - A\mathbf{x}^{(0)}$ 
3:  $\mathbf{p}^{(0)} \leftarrow \mathbf{r}^{(0)}$ 
4:  $k \leftarrow 0$ 

```

```

5: while stopping criteria are not satisfied do
6:    $\alpha^{(k)} \leftarrow (\mathbf{r}^{(k)T} \mathbf{p}^{(k)}) / (\mathbf{p}^{(k)T} A \mathbf{p}^{(k)})$ 
7:    $\mathbf{x}^{(k+1)} \leftarrow \mathbf{x}^{(k)} + \alpha^{(k)} \mathbf{p}^{(k)}$ 
8:    $\mathbf{r}^{(k+1)} \leftarrow \mathbf{b} - A \mathbf{x}^{(k+1)}$ 
9:    $\beta^{(k+1)} \leftarrow -(\mathbf{p}^{(k)T} A \mathbf{r}^{(k+1)}) / (\mathbf{p}^{(k)T} A \mathbf{p}^{(k)})$ 
10:   $\mathbf{p}^{(k+1)} \leftarrow \mathbf{r}^{(k+1)} + \beta^{(k+1)} \mathbf{p}^{(k)}$ 
11:   $k \leftarrow k + 1$ 
12: end while
13:  $\hat{\mathbf{x}} \leftarrow \mathbf{x}^{(k)}$ 
14: return :  $\hat{\mathbf{x}}$ 

```

**Algorithm 3.4** (Conjugate Gradient Method).

```

1:  $\mathbf{x}^{(0)}$  given
2:  $\mathbf{r}^{(0)} \leftarrow \mathbf{b} - A \mathbf{x}^{(0)}$ 
3:  $\mathbf{p}^{(0)} \leftarrow \mathbf{r}^{(0)}$ 
4:  $k \leftarrow 0$ 
5: while stopping criteria are not satisfied do
6:    $\mathbf{z}^{(k)} \leftarrow A \mathbf{p}^{(k)}$ 
7:    $\alpha^{(k)} \leftarrow (\mathbf{r}^{(k)T} \mathbf{p}^{(k)}) / (\mathbf{p}^{(k)T} \mathbf{z}^{(k)})$ 
8:    $\mathbf{x}^{(k+1)} \leftarrow \mathbf{x}^{(k)} + \alpha^{(k)} \mathbf{p}^{(k)}$ 
9:    $\mathbf{r}^{(k+1)} \leftarrow \mathbf{r}^{(k)} - \alpha^{(k)} \mathbf{z}^{(k)}$ 
10:   $\beta^{(k+1)} \leftarrow -(\mathbf{r}^{(k+1)T} \mathbf{z}^{(k)}) / (\mathbf{p}^{(k)T} \mathbf{z}^{(k)})$ 
11:   $\mathbf{p}^{(k+1)} \leftarrow \mathbf{r}^{(k+1)} + \beta^{(k+1)} \mathbf{p}^{(k)}$ 
12:   $k \leftarrow k + 1$ 
13: end while
14:  $\hat{\mathbf{x}} \leftarrow \mathbf{x}^{(k)}$ 
15: return :  $\hat{\mathbf{x}}$ 

```

### 3.2.1 Preconditioning

The convergence of the CG is characterized by the following property:

$$\|\mathbf{x}^{(k)} - \mathbf{x}^*\|_A \leq 2 \left( \frac{\sqrt{\kappa_2(A)} - 1}{\sqrt{\kappa_2(A)} + 1} \right)^k \|\mathbf{x}^{(0)} - \mathbf{x}^*\|_A, \quad \forall k \geq 0, \quad (25)$$

where  $\|\cdot\|_A$  is the *energy norm* such that  $\|\mathbf{x}\|_A := \sqrt{\mathbf{x}^T A \mathbf{x}}$ , and  $\kappa_2(A) := \|A\|_2 \|A^{-1}\|_2 = \lambda_{\max}/\lambda_{\min}$  ( $\lambda$  denotes eigenvalues of  $A$ ).

**Observation:** the more ill-conditioned  $A$ , the larger is the upper bound of  $\|\mathbf{x}^{(k)} - \mathbf{x}^*\|_A$  and, therefore, the slower can be the convergence. Nonetheless, the upper bound in (25) is lower than the one in (22).

Given the above observation, we observe that a method to reduce the effects of an ill-conditioned matrix exists. The main idea is to introduce a variable substitution

$$\mathbf{x} = C^{-1} \mathbf{y}. \quad (26)$$

Using (26), we have that (16) changes into  $J(\mathbf{y}) = \frac{1}{2} \mathbf{y}^T C^{-T} A C^{-1} \mathbf{y} - \mathbf{y}^T C^{-T} \mathbf{b}$ , and we look for the solution of the linear system  $C^{-T} A C^{-1} \mathbf{y} = C^{-T} \mathbf{b}$ . Therefore, with a proper choice of  $C$  we can reduce the condition number of the new matrix and speed up the convergence of the CG method.

Typically, the “best” *preconditioner* (i.e., the matrix  $C$ ) does not exist. In general, it depends on the nature of the matrix  $A$ . Nonetheless, a canonical choice can be the matrix  $\tilde{L}^T$  returned by the *incomplete Cholesky* method.

The incomplete Cholesky method performs an incomplete Cholesky factorization such that it returns a *sparse* lower triangular matrix  $\tilde{L}$  such that  $\tilde{L}^T \approx R$ , where  $R$  is the upper triangular

matrix of the Cholesky factorization (see Section 1.1). The choice of  $C = \tilde{L}^T$  is motivated by the fact that the more  $\tilde{L}^T \approx R$ , the more  $C^{-T}AC^{-1} \approx \mathbb{I}_n$  and, therefore, the problem is more stable, because:

$$\mathbb{I}_n R\mathbf{x} \approx C^{-T}AC^{-1}\mathbf{y} = C^{-T}\mathbf{b} \approx R^{-T}\mathbf{b} \quad (27)$$

Of course, the more  $\tilde{L}^T \approx R$ , the more the density of  $\tilde{L}$ , losing the advantages of sparsity.

Such kind of preconditioning is called *two-sided* or *split* preconditioning (there are also *left/right* preconditioning methods in the literature).

### 3.3 Other Iterative Methods

We point the attention of the reader on the characteristics of the matrix  $A$  we used in this section. We have always assumed  $A$  *symmetric* and *positive definite* (at most, we can relax the hypotheses to *positive semi-definiteness*). Then, if  $A$  does not satisfy these properties, the gradient and CG methods cannot be applied.

Other iterative methods, can be for example the *generalized minimal residual method* (GMRES), where the preconditioning can be performed exploiting an incomplete LU factorization.

### 3.4 Exercises

From now on, we will write the text of the exercises for Matlab users. Nonetheless, you can use Python instead.

**Exercise 3.1** (Gradient Method). Write a function called `gradient_linsys` that implement in Matlab the Algorithm 3.2 using a relative residual as stopping criterium.

Test your function on the linear system defined by the variables inside the file `lab03_sparse_linsys.mat`.

**Exercise 3.2** (Conjugate Gradient Method). Write a function called `cg_linsys` that implement in Matlab the Algorithm 3.4 using a relative residual as stopping criterium.

Test your function on the linear system defined by the variables inside the file `lab03_sparse_linsys.mat`.

**Exercise 3.3.** Write a script where:

1. Generate two sparse tridiagonal matrices (see `spdiags` function)  $A_1, A_2 \in \mathbb{R}^{n \times n}$  (say  $n = 1000$ ) such that

$$A_i = \begin{bmatrix} \alpha_i & -1 & & & \\ -1 & \ddots & \ddots & & \\ & \ddots & \ddots & \ddots & \\ & & \ddots & \ddots & -1 \\ & & & -1 & \alpha_i \end{bmatrix}, \quad \alpha_1 = 4, \alpha_2 = 2.$$

2. Compute and visualize  $k_2(A_1), k_2(A_2)$ ;
3. For each  $\tau = 10^{-4}, \dots, 10^{-10}$ , solve the linear systems  $A_i \mathbf{x} = \mathbf{b}_i$ , with exact solution  $\mathbf{x}_i^* = (1, \dots, 1) \in \mathbb{R}^n$ ,  $i = 1, 2$  (i.e., build  $\mathbf{b}_i$  as in Exercise 1.1, item 3).

In particular, solve the linear systems with `gradient_linsys` of Exercise 3.1, with `cg_linsys` of Exercise 3.2, and with the `pcg` matlab function, both with and without preconditioning (see `ichol` function).

4. *Plot the number of iterations used by the methods for solving the linear systems, varying the tolerance values.*

## 4 Steepest Descent

Let the function  $f : \mathbb{R}^n \rightarrow \mathbb{R}$  be given. The Steepest Descent (SD) method is an iterative optimization method that can be interpreted as the nonlinear generalization of the Gradient method; SD starts from a given vector  $\mathbf{x}^{(0)} \in \mathbb{R}^n$ , and computes a sequence of vectors  $\{\mathbf{x}^{(k)}\}_{k \in \mathbb{N}}$  characterized by

$$\mathbf{x}^{(k+1)} = \mathbf{x}^{(k)} + \alpha^{(k)} \mathbf{p}^{(k)}, \quad \forall k \geq 0, \quad (28)$$

where the descent direction  $\mathbf{p}^{(k)}$  is the steepest one, i.e.  $\mathbf{p}^{(k)} = -\nabla f(\mathbf{x}^{(k)})$ , and the step length factor  $\alpha \in \mathbb{R}^+$ .

**Attention:** Contrary to the Gradient method, we have not a formula for computing the optimal step length  $\alpha^{(k)}$  (because  $f$  is a generic differentiable function). The choice of  $\alpha^{(k)}$  is problem-dependent and different strategies exist to select the optimal one.

**N.B.:** In this laboratory, we will use a fixed value  $\alpha = \alpha^{(k)}$  for each step.

### 4.1 Exercises

The text of the exercises is for Matlab users. Nonetheless, you can use Python instead.

**Exercise 4.1** (Steepest Descent). *Write a Matlab function `steepest_descent.m` that implements the steepest descent optimization method, given:*

- **`x0`**: a column vector of  $n$  elements representing the starting point for the optimization method;
- **`f`**: a function handle variable that, for each column vector  $\mathbf{x} \in \mathbb{R}^n$ , returns the value  $f(\mathbf{x})$ , where  $f : \mathbb{R}^n \rightarrow \mathbb{R}$  is the loss function that have to be minimized;
- **`gradf`**: a function handle variable that, for each column vector  $\mathbf{x} \in \mathbb{R}^n$ , returns the value  $\nabla f(\mathbf{x})$  as a column vector, where  $\nabla f : \mathbb{R}^n \rightarrow \mathbb{R}^n$  is the gradient of  $f$ ;
- **`alpha`**: a real scalar value characterizing the step length of the optimization method (fixed, for simplicity);
- **`kmax`**: an integer scalar value characterizing the maximum number of iterations of the method;
- **`tolgrad`**: a real scalar value characterizing the tolerance with respect to the norm of the gradient in order to stop the method.

The outputs of the function must be:

- **`xk`**: the last vector  $\mathbf{x}^{(k)} \in \mathbb{R}^n$  computed by the optimization method before it stops;
- **`fk`**: the value  $f(\mathbf{x}^{(k)})$ ;
- **`gradfk_norm`**: the euclidean norm of  $\nabla f(\mathbf{x}^{(k)})$ ;
- **`k`**: index value of the last step executed by the optimization method before stopping;
- **`xseq`**: a matrix/vector in  $\mathbb{R}^{n \times k}$  such that each column  $j$  is the  $j$ -th vector  $\mathbf{x}^{(j)} \in \mathbb{R}^n$  generated by the iterations of the method.

Once you have written the function, test it using the data `x0`, `f`, `gradf`, `alpha`, `kmax`, `tolgrad`, inside the file `test_functions2.mat`

**Exercise 4.2** (Steepest Descent Path Visualization). *Given the function of the previous exercise, write a Matlab script where you test this function using the 2-dimensional functions inside the file `test_functions2.mat` (see Section B.1). In particular, for each 2-dimensional function in the `.mat` file:*

- *run the SD procedure by varying  $\alpha = 1e-1, 1e-2, 1e-3, 1e-4$ ;*
- *repeat the previous item, but using the Finite Differences (of your choice) to compute the gradient of the functions, varying  $h$  among the values  $h = 1e-8, 1e-6, 1e-4$ ;*
- *For each procedure generated with the previous two steps, collect:*
  - *the number of steps for reaching the convergence (or a stopping criterion);*
  - *the norm of the gradient at the last step;*
  - *the coordinates of the minimizer  $\hat{\mathbf{x}}$  (just because we are in 2D) and the value  $f(\hat{\mathbf{x}})$ ;*
- *plot a top view of the loss  $f$  using the Matlab function `contour`, together with the sequence  $\mathbf{xseq}$  in  $\mathbb{R}^2$ ;*
- *the surface of the loss  $f$  using the Matlab function `surf`, together with the function values of the sequence  $\mathbf{xseq}$  in  $\mathbb{R}^3$  (i.e., coordinates plus function values). See the `meshgrid` function for preparing the plot domain. You can use the function `f_meshgrid` for evaluating the function on the output matrices of `meshgrid`. For any doubts, see <https://www.mathworks.com/help/matlab/ref/surf.html>.*

**Exercise 4.3** ( $n$ -Dimensional Steepest Descent). *Repeat the previous exercise (except for the plots) with respect to the  $n$ -dimensional function in `test_funcs_ndim.mat`. Run the test for different values of  $n$ , e.g.:  $n = 10, 100, 1000$ .*

## 5 Steepest Descent with Backtracking

Let the function  $f : \mathbb{R}^n \rightarrow \mathbb{R}$  be given. In the previous laboratory we have seen that the steepest descent method is an iterative optimization method that, starting from a given vector  $\mathbf{x}^{(0)} \in \mathbb{R}^n$ , computes a sequence of vectors  $\{\mathbf{x}^{(k)}\}_{k \in \mathbb{N}}$  characterized by

$$\mathbf{x}^{(k+1)} = \mathbf{x}^{(k)} + \alpha \mathbf{p}^{(k)}, \quad \forall k \geq 0, \quad (29)$$

where the descent direction  $\mathbf{p}^{(k)}$  is the steepest one, i.e.  $\mathbf{p}^{(k)} = -\nabla f(\mathbf{x}^{(k)})$ , and the step length factor  $\alpha \in \mathbb{R}^+$ .

### 5.1 Backtracking

Let the function  $f : \mathbb{R}^n \rightarrow \mathbb{R}$  be given. The backtracking strategy for an iterative optimization method consists of looking for a value  $\alpha^{(k)}$  satisfying the Armijo condition at each step  $k$  of the method, i.e.

$$f(\underbrace{\mathbf{x}^{(k+1)}}_{\mathbf{x}^{(k)} + \alpha^{(k)} \mathbf{p}^{(k)}}) \leq f(\mathbf{x}^{(k)}) + c_1 \alpha^{(k)} \nabla f(\mathbf{x}^{(k)})^T \mathbf{p}^{(k)}, \quad (30)$$

where  $c_1 \in (0, 1)$  (typically, the standard choice is  $c_1 = 10^{-4}$ ).

We recall that the Armijo condition suggests that a “good”  $\alpha^{(k)}$  is such that you have a sufficient decrease in  $f$  and, moreover, the function value at the new point  $f(\mathbf{x}^{(k+1)}) = f(\mathbf{x}^{(k)} + \alpha^{(k)} \mathbf{p}^{(k)})$  is under the “reduced tangent hyperplane” of  $f$  at  $\mathbf{x}^{(k)}$ .

To better explain the Armijo condition, we look at the function  $\phi(\alpha) := f(\mathbf{x}^{(k)} + \alpha \mathbf{p}^{(k)})$ , such that  $\phi'(\alpha) = \nabla f(\mathbf{x}^{(k)} + \alpha \mathbf{p}^{(k)})^T \mathbf{p}^{(k)}$  and  $\phi'(0) = \nabla f(\mathbf{x}^{(k)})^T \mathbf{p}^{(k)}$  (see Figure 4).

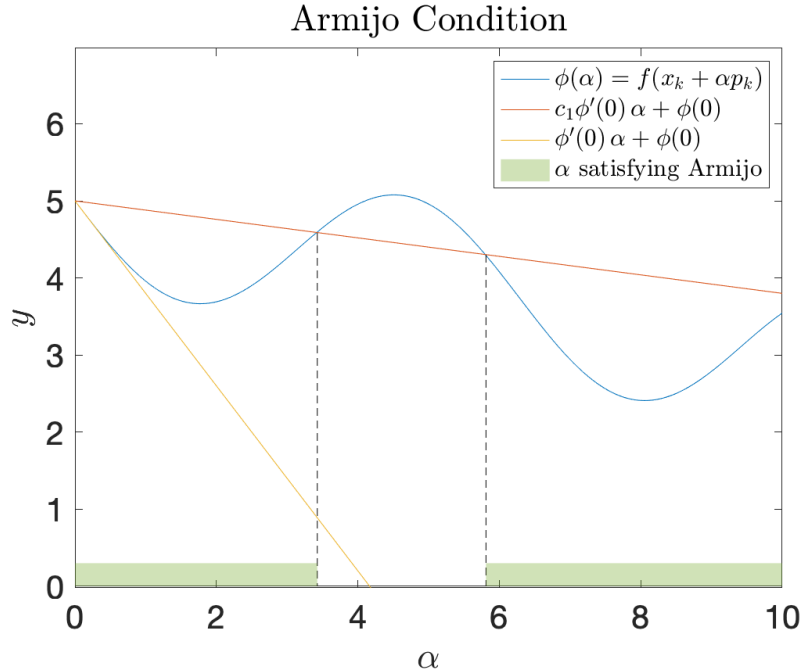


Figure 4: Example of the Armijo condition.

The backtracking strategy is an *iterative process* that looks for this value  $\alpha^{(k)}$ . Given a factor  $\rho \in (0, 1)$  and a starting value  $\alpha_0^{(k)}$  for  $\alpha^{(k)}$ , we decrease iteratively  $\alpha_0^{(k)}$ , multiplying it by  $\rho$ , until the Armijo condition is satisfied. Then  $\alpha^{(k)} = \rho^{t_k} \alpha_0^{(k)}$ , for a  $t_k \in \mathbb{N}$ , if it satisfies Armijo but  $\rho^{t_k-1} \alpha_0^{(k)}$  does not.



**Remark 5.1** (Few things to keep in mind).

1. The Armijo condition is always satisfied for extremely small values of  $\alpha$ . Then, it is not enough to ensure that the algorithm makes reasonable progress; indeed, if  $\alpha$  is too small, unacceptably short steps are taken (see item 3 below).
2. For simplicity, we consider  $\rho$  as a fixed parameter, but it can be chosen using already available information, changing with the iterations;
3. Other conditions could be imposed to guarantee that not too-short steps are taken (e.g., Wolfe conditions<sup>1</sup>), but they are not practical to be implemented. Practical implementations, instead of imposing a second condition, frequently use the backtracking strategy; for example, a proper choice of  $\alpha_0^{(k)}$ ,  $\rho$ , and the maximum number  $T$  of backtracking steps can guarantee that  $\rho^T \alpha_0^{(k)} \geq \epsilon$ , where  $\epsilon$  is the minimum step-length acceptable.
4. The choice of  $\alpha_0^{(k)}$  is problem-dependent and/or method-dependent.

## 5.2 Exercises

The text of the exercises is for Matlab users. Nonetheless, you can use Python instead.

**Exercise 5.1** (Steepest Descent with Backtracking). *Write a Matlab function `steepest_desc_bcktrck.m` that implements the steepest descent optimization method with the backtracking strategy, given:*

- ***x0***: a column vector of  $n$  elements representing the starting point for the optimization method;
- ***f***: a function handle variable that, for each column vector  $\mathbf{x} \in \mathbb{R}^n$ , returns the value  $f(\mathbf{x})$ , where  $f : \mathbb{R}^n \rightarrow \mathbb{R}$  is the loss function that have to be minimized;
- ***gradf***: a function handle variable that, for each column vector  $\mathbf{x} \in \mathbb{R}^n$ , returns the value  $\nabla f(\mathbf{x})$  as a column vector, where  $\nabla f : \mathbb{R}^n \rightarrow \mathbb{R}^n$  is the gradient of  $f$ ;
- ***alpha0***: a real scalar value characterizing the starting value for the backtracking (fixed, for simplicity);
- ***kmax***: an integer scalar value characterizing the maximum number of iterations of the method;
- ***tolgrad***: a real scalar value characterizing the tolerance with respect to the norm of the gradient in order to stop the method.
- ***c1***: the factor  $c_1$  for the Armijo condition that must be a scalar in  $(0, 1)$ ;
- ***rho***: factor less than 1, used to reduce  $\alpha$  (fixed, for simplicity);
- ***btmax***: maximum number of steps allowed to update  $\alpha$  during the backtracking strategy.

The outputs of the function must be:

- ***xk***: the last vector  $\mathbf{x}^{(k)} \in \mathbb{R}^n$  computed by the optimization method before it stops;
- ***fk***: the value  $f(\mathbf{x}^{(k)})$ ;
- ***gradfk\_norm***: the euclidean norm of  $\nabla f(\mathbf{x}^{(k)})$ ;

---

<sup>1</sup>i.e., Armijo condition and curvature condition  $\nabla f(\mathbf{x}_{k+1})^T \mathbf{p}_k \geq c_2 \nabla f(\mathbf{x}_k)^T \mathbf{p}_k$ ,  $c_2 \in (c_1, 1)$ .

- **k**: index value of the last step executed by the optimization method before stopping;
- **xseq**: a matrix/vector in  $\mathbb{R}^{n \times k}$  such that each column  $j$  is the  $j$ -th vector  $\mathbf{x}^{(j)} \in \mathbb{R}^n$  generated by the iterations of the method.
- **btseq**: row vector in  $\mathbb{R}^k$  such that the  $j$ -th element is the number of backtracking iterations done at the  $j$ -th step of the steepest descent.

Extra options:

- implement the procedure such that it stops if the backtracking strategy cannot find  $\alpha^{(k)}$  satisfying the Armijo condition;
- **flag**: an extra output variable for the function containing a string that is a message. The message explains the result (e.g. "Procedure stopped in  $k$  steps, with gradient norm  $\varepsilon$ " and/or "Procedure stopped because...").

Once you have written the function, test it using the data **x0**, **f**, **gradf**, **alpha0**, **kmax**, **tolgrad**, **c1**, **rho**, **btmax**, inside the file **test\_functions2.mat**

**Suggestion:** copy the code of the function of Exercise 4.1 and modify it adding the backtracking.

**Exercise 5.2** (Steepest Descent with Backtracking Path Visualization). Given the function of the previous exercise, write a Matlab script where you test this function using the 2-dimensional functions inside the file **test\_functions2.mat** (see Section B.1). In particular, for each 2-dimensional function in the **.mat** file:

- run the SD + backtracking procedure by varying  $\rho = 0.9, 0.75, 0.5, 0.25, 0.1$ , with fixed  $\alpha_0^{(k)} = 1$  and  $c_1 = 1e-4$ , **btmax** = 50;
- repeat the previous item, but using the Finite Differences (of your choice) to compute the gradient of the functions, varying  $h$  among the values  $h = 1e-8, 1e-6, 1e-4$ ;
- For each procedure generated with the previous two steps, collect:
  - the number of steps for reaching the convergence (or a stopping criterion);
  - the norm of the gradient at the last step;
  - the coordinates of the minimizer  $\hat{\mathbf{x}}$  (just because we are in 2D) and the value  $f(\hat{\mathbf{x}})$ ;
- plot a top view of the loss  $f$  using the Matlab function **contour**, together with the sequence **xseq** in  $\mathbb{R}^2$ ;
- the surface of the loss  $f$  using the Matlab function **surf**, together with the function values of the sequence **xseq** in  $\mathbb{R}^3$  (i.e., coordinates plus function values). See the **meshgrid** function for preparing the plot domain. You can use the function **f\_meshgrid** for evaluating the function on the output matrices of **meshgrid**. For any doubts, see <https://www.mathworks.com/help/matlab/ref/surf.html>.
- Make a comparison of the results obtained in this exercise with the results obtained in Exercise 4.2.

**Exercise 5.3** ( $n$ -Dimensional Steepest Descent with Backtracking). Repeat the previous exercise (except for the plots) with respect to the  $n$ -dimensional function in **test\_funcs\_ndim.mat**. Run the test for different values of  $n$ , e.g.:  $n = 10, 100, 1000$ . Make a comparison of the results obtained in this exercise with the results obtained in Exercise 4.3.

## 6 Newton Method with Backtracking

Let the function  $f : \mathbb{R}^n \rightarrow \mathbb{R}$  be given. The Newton descent method is an iterative optimization method that, starting from a given vector  $\mathbf{x}^{(0)} \in \mathbb{R}^n$ , computes a sequence of vectors  $\{\mathbf{x}^{(k)}\}_{k \in \mathbb{N}}$  characterized by

$$\mathbf{x}^{(k+1)} = \mathbf{x}^{(k)} + \alpha \mathbf{p}^{(k)}, \quad \forall k \geq 0, \quad (31)$$

where the descent direction  $\mathbf{p}^{(k)}$  is solution of the linear system

$$Hf(\mathbf{x}^{(k)})\mathbf{p} = -\nabla f(\mathbf{x}^{(k)}), \quad (32)$$

and  $\nabla f$ ,  $Hf$  are the gradient and the Hessian matrix of  $f$ , respectively. Typically,  $\alpha = 1$  almost always (see PROs and CONs in the next subsection).

### 6.1 Recalling the Motivations of Newton

We can approximate  $f(\mathbf{x} + \mathbf{p})$ , with a Taylor expansion, as a quadratic model, i.e.:

$$f(\mathbf{x} + \mathbf{p}) \simeq f(\mathbf{x}) + \mathbf{p}^T \nabla f(\mathbf{x}) + \frac{1}{2} \mathbf{p}^T Hf(\mathbf{x}) \mathbf{p} =: m_{f,\mathbf{x}}(\mathbf{p}). \quad (33)$$

Assuming that  $Hf(\mathbf{x})$  is Positive Semidefinite (PSD), for each  $\mathbf{p} \in \mathbb{R}^n$ , we have that  $\mathbf{p}^T Hf(\mathbf{x}) \mathbf{p} \geq 0$  and, therefore,  $m_{f,\mathbf{x}}(\mathbf{p})$  is *convex*. Then, we can compute the  $\mathbf{p}^*$  that minimizes  $m_{f,\mathbf{x}}(\mathbf{p})$ , i.e. the  $\mathbf{p}^*$  such that

$$\underbrace{\nabla f(\mathbf{x}) + Hf(\mathbf{x})\mathbf{p}^*}_{=\nabla_{\mathbf{p}} m_{f,\mathbf{x}}(\mathbf{p}^*)} = \mathbf{0}. \quad (34)$$

In other words,  $\mathbf{p}^*$  is solution of the linear system (32) and is a *descent direction* for  $f(\mathbf{x})$ .

In the end, we recall the PROs and CONs of the Newton method:

- (+) Under the proper assumptions of having  $Hf(\mathbf{x}_k)$  PD,  $\alpha = 1$ , and  $\mathbf{x}^{(0)}$  “good” starting point, the Newton method has fast (quadratic) rate of convergence;
- (−) Computationally more expensive than Steepest Descent (computation/storage of  $Hf(\mathbf{x}^{(k)})$  and computation of  $\mathbf{p}^{(k)}$  solving (32));
- (−)  $\mathbf{p}^{(k)}$  is a descent direction only if  $Hf(\mathbf{x}^{(k)})$  is PSD  $\Rightarrow$  Sometimes is useful to work with a “corrected” Hessian matrix, if  $Hf(\mathbf{x}_k)$  is not PD/PSD; i.e., we can use a PD matrix  $B_k := Hf(\mathbf{x}_k) + \text{Correction}$  (see the *Modified Newton* method in [2]).

### 6.2 Backtracking and Newton

In this laboratory, we focus on the implementation of the Newton method paired with a backtracking strategy (see Section 5.1). Therefore, we point the reader to Remark 5.1 for recalling some observations related to Backtracking; furthermore, in the case of the Newton method, we update the last item of Remark 5.1 with the following one:

- The choice of  $\alpha_0^{(k)}$  is problem-dependent and/or method-dependent, but it is *crucial* to choose  $\alpha_0^{(k)} = 1$  in Newton/Quasi-Newton methods for (possibly) get the second-order rate of convergence.

### 6.3 Exercises

The text of the exercises is for Matlab users. Nonetheless, you can use Python instead.

**Exercise 6.1** (Newton with Backtracking). Write a Matlab function `newton_bcktrck.m` that implements the Newton optimization method with the backtracking strategy, given:

- **`x0`**: a column vector of  $n$  elements representing the starting point for the optimization method;
- **`f`**: a function handle variable that, for each column vector  $\mathbf{x} \in \mathbb{R}^n$ , returns the value  $f(\mathbf{x})$ , where  $f : \mathbb{R}^n \rightarrow \mathbb{R}$  is the loss function that have to be minimized;
- **`gradf`**: a function handle variable that, for each column vector  $\mathbf{x} \in \mathbb{R}^n$ , returns the value  $\nabla f(\mathbf{x})$  as a column vector, where  $\nabla f : \mathbb{R}^n \rightarrow \mathbb{R}^n$  is the gradient of  $f$ ;
- **`Hessf`**: a function handle variable that, for each column vector  $\mathbf{x} \in \mathbb{R}^n$ , returns the value  $Hf(\mathbf{x})$ , where  $Hf : \mathbb{R}^n \rightarrow \mathbb{R}^{n \times n}$  is the Hessian of  $f$ ;
- **`kmax`**: an integer scalar value characterizing the maximum number of iterations of the method;
- **`tolgrad`**: a real scalar value characterizing the tolerance with respect to the norm of the gradient in order to stop the method.
- **`c1`**: the factor  $c_1$  for the Armijo condition that must be a scalar in  $(0, 1)$ ;
- **`rho`**: factor less than 1, used to reduce  $\alpha$  (fixed, for simplicity);
- **`btmax`**: maximum number of steps allowed to update  $\alpha$  during the backtracking strategy.

The outputs of the function must be:

- **`xk`**: the last vector  $\mathbf{x}^{(k)} \in \mathbb{R}^n$  computed by the optimization method before it stops;
- **`fk`**: the value  $f(\mathbf{x}^{(k)})$ ;
- **`gradfk_norm`**: the euclidean norm of  $\nabla f(\mathbf{x}^{(k)})$ ;
- **`k`**: index value of the last step executed by the optimization method before stopping;
- **`xseq`**: a matrix/vector in  $\mathbb{R}^{n \times k}$  such that each column  $j$  is the  $j$ -th vector  $\mathbf{x}^{(j)} \in \mathbb{R}^n$  generated by the iterations of the method.
- **`btseq`**: row vector in  $\mathbb{R}^k$  such that the  $j$ -th element is the number of backtracking iterations done at the  $j$ -th step of the steepest descent.

Extra options:

- implement the procedure such that it stops if the backtracking strategy cannot find  $\alpha^{(k)}$  satisfying the Armijo condition;
- **`flag`**: an extra output variable for the function containing a string that is a message. The message explains the result (e.g. “Procedure stopped in  $k$  steps, with gradient norm  $\varepsilon$ ” and/or “Procedure stopped because...”).

Once you have written the function, test it using the data inside the file `test_functions2.mat`

**Important:** it is better if you solve the linear system (32) by using the `pcg` Matlab function, and not the “backslash”. This will be useful for implementations in future laboratories.

**Suggestion:** copy the code of the function of Exercise 5.1 and modify it where it is necessary.

**Exercise 6.2** (Newton with Backtracking Path Visualization). *Given the function of the previous exercise, write a Matlab script where you test this function using the 2-dimensional functions inside the file `test_functions2.mat` (see Section B.1). In particular, for each 2-dimensional function in the `.mat` file:*

- *run the Newton + backtracking procedure by varying  $\rho = 0.9, 0.75, 0.5, 0.25, 0.1$ , with fixed  $c_1 = 1e-4$ , `btmax` = 50;*
- *repeat the previous item, but using the Finite Differences (of your choice) to compute the gradient and/or the Hessian of the functions, varying  $h$  among the values  $h = 1e-8, 1e-6, 1e-4$ ;*
- *For each procedure generated with the previous two steps, collect:*
  - *the number of steps for reaching the convergence (or a stopping criterion);*
  - *the norm of the gradient at the last step;*
  - *the coordinates of the minimizer  $\hat{\mathbf{x}}$  (just because we are in 2D) and the value  $f(\hat{\mathbf{x}})$ ;*
- *plot a top view of the loss  $f$  using the Matlab function `contour`, together with the sequence  $\mathbf{xseq}$  in  $\mathbb{R}^2$ ;*
- *the surface of the loss  $f$  using the Matlab function `surf`, together with the function values of the sequence  $\mathbf{xseq}$  in  $\mathbb{R}^3$  (i.e., coordinates plus function values). See the `meshgrid` function for preparing the plot domain. You can use the function `f_meshgrid` for evaluating the function on the output matrices of `meshgrid`. For any doubts, see <https://www.mathworks.com/help/matlab/ref/surf.html>.*
- *Make a comparison of the results obtained in this exercise with the results obtained in Exercise 4.2 and Exercise 5.2.*

**Exercise 6.3** ( $n$ -Dimensional Newton with Backtracking). *Repeat the previous exercise (except for the plots) with respect to the  $n$ -dimensional function in `test_funcs_ndim.mat`. Run the test for different values of  $n$ , e.g.:  $n = 10, 100, 1000$ . Make a comparison of the results obtained in this exercise with the results obtained in Exercise 4.3 and Exercise 5.3.*

## 7 Inexact Newton and Orders of Convergence

Sometimes the computation of the Newton descent direction  $\mathbf{p}_N^{(k)}$ , solving the linear system

$$Hf(\mathbf{x}^{(k)})\mathbf{p} = -\nabla f(\mathbf{x}^{(k)}), \quad (35)$$

can be quite expensive. Therefore, in order to save computational efforts, instead of computing  $\mathbf{p}_N^{(k)}$ , we compute an “approximated descent direction”  $\mathbf{p}_{IN}^{(k)} \approx \mathbf{p}_N^{(k)}$ .

### 7.1 Inexact Descent Direction and Forcing Terms

Assuming the usage of an iterative method to solve (35), at the  $k$ -th step the iterative solver returns<sup>2</sup> a vector  $\mathbf{p}^{(k)*}$  with residual such that

$$\underbrace{\| Hf(\mathbf{x}^{(k)})\mathbf{p}^{(k)*} + \nabla f(\mathbf{x}^{(k)}) \|}_{\text{residual}} \leq \varepsilon, \quad (36)$$

where  $\varepsilon > 0$  a tolerance used for the iterative method.

Then, instead of using a small and fixed  $\varepsilon$ , the Inexact Newton method consists of using an adaptive tolerance

$$\varepsilon_k := \underbrace{\eta_k}_{\text{forcing terms}} \| \nabla f(\mathbf{x}^{(k)}) \| \quad (37)$$

that is relatively “large” during the first optimization steps; i.e., we use a large tolerance when we assume to be far from the optimum.

In general, under *local and regularity assumptions*, the most used forcing term sequences are the following:

- $\eta_k = 0.5$  fixed (*linear* convergence);
- $\eta_k = \min(0.5, \sqrt{\| \nabla f(\mathbf{x}^{(k)}) \|}) \xrightarrow{k \rightarrow \infty} 0$  (*superlinear* convergence);
- $\eta_k = \min(0.5, \| \nabla f(\mathbf{x}^{(k)}) \|) \in \mathcal{O}(\| \nabla f(\mathbf{x}^{(k)}) \|)$  (*quadratic* convergence).

**N.B.:** the order of convergence also depends on the iterative solver; e.g. some exceptions are observable in the case where the solver, despite the larger tolerance, returns always/often a solution  $\mathbf{p}_{IN}^{(k)} \approx \mathbf{p}_N^{(k)}$  (i.e. with residual  $\sim \varepsilon$ ).

**ATTENTION:** The *pcg* Matlab function uses the *relative residual*  $\| A\mathbf{x} - \mathbf{b} \| / \| \mathbf{b} \|$  for its stopping criteria and not the *residual*  $\| A\mathbf{x} - \mathbf{b} \|$  (see Section 3). Then, pay attention to defining the tolerance! Indeed, in order to have  $\| Hf(\mathbf{x}^{(k)})\mathbf{p}^{(k)} - \nabla f(\mathbf{x}^{(k)}) \| \leq \eta_k \| \nabla f(\mathbf{x}^{(k)}) \|$ , you should set the relative tolerance  $\| Hf(\mathbf{x}^{(k)})\mathbf{p}^{(k)} - \nabla f(\mathbf{x}^{(k)}) \| / \| \nabla f(\mathbf{x}^{(k)}) \| \leq \dots$  *exercise...*

### 7.2 Orders of Convergence

In this subsection, we recall the definitions concerning the order of convergence  $q$  of a sequence  $\{\mathbf{x}^{(k)}\}_{k \in \mathbb{N}}$  converging to  $\mathbf{x}^*$  and how to estimate it.

**Definition 7.1** (Orders of Convergence). *Let  $\{\mathbf{x}^{(k)}\}_{k \in \mathbb{N}} \subset \mathbb{R}^n$  be a sequence converging to  $\mathbf{x}^* \in \mathbb{R}^n$ . Let  $\mathbf{e}^{(k)}$  denotes the error of  $\mathbf{x}^{(k)}$  in approximating  $\mathbf{x}^*$ , i.e.,  $\mathbf{e}^{(k)} := \mathbf{x}^{(k)} - \mathbf{x}^*$ . Then, we have that:*

---

<sup>2</sup>if the maximum iterations have not been reached.

- the convergence is Q-linear if exists  $M \in (0, 1)$  such that

$$\frac{\| \mathbf{e}^{(k+1)} \|}{\| \mathbf{e}^{(k)} \|} \leq M \quad (38)$$

for each  $k$  sufficiently large.

- the convergence is Q-superlinear if

$$\lim_{k \rightarrow +\infty} \frac{\| \mathbf{e}^{(k+1)} \|}{\| \mathbf{e}^{(k)} \|} = 0. \quad (39)$$

- for each  $q > 1$ , the convergence is of Q-order  $q$  if exists  $M > 0$  such that

$$\frac{\| \mathbf{e}^{(k+1)} \|}{\| \mathbf{e}^{(k)} \|^q} \leq M \quad (40)$$

for each  $k$  sufficiently large.

The “Q” letter in the order of convergence is for “quotient” and can be omitted.

**N.B.:** Obviously, if a sequence converges with order  $q$ , then it converges also with order  $r$ , for each  $r < q$ .

Given Definition 7.1, we can describe a recipe for estimating the order of convergence of a sequence. In particular, we have that the order of convergence  $q$  of a sequence  $\{\mathbf{x}^{(k)}\}_{k \in \mathbb{N}}$  converging to  $\mathbf{x}^*$  is

$$q \approx \frac{\log \left( \frac{\| \mathbf{e}^{(k+1)} \|}{\| \mathbf{e}^{(k)} \|} \right)}{\log \left( \frac{\| \mathbf{e}^{(k)} \|}{\| \mathbf{e}^{(k-1)} \|} \right)} \quad \text{for } k \text{ sufficiently large,} \quad (41)$$

because it holds both  $\| \mathbf{e}^{(k+1)} \| \approx \| \mathbf{e}^{(k)} \|^q$  and  $\| \mathbf{e}^{(k)} \| \approx \| \mathbf{e}^{(k-1)} \|^q$ ; then,  $\| \mathbf{e}^{(k+1)} \| / \| \mathbf{e}^{(k)} \| \approx (\| \mathbf{e}^{(k)} \| / \| \mathbf{e}^{(k-1)} \|)^q$ , which implies  $\log(\| \mathbf{e}^{(k+1)} \| / \| \mathbf{e}^{(k)} \|) \approx q \log(\| \mathbf{e}^{(k)} \| / \| \mathbf{e}^{(k-1)} \|)$  and, as a consequence, the estimate (41) holds.

However, the exact value of  $\mathbf{x}^*$  is rarely known; nonetheless, for  $k$  sufficiently large, we can approximate  $\mathbf{e}^{(k)}$  with  $\hat{\mathbf{e}}^{(k)} := \mathbf{x}^{(k)} - \mathbf{x}^{(k-1)}$ . Therefore, we can further approximate the order of convergence with the following formula:

$$q \approx \frac{\log \left( \frac{\| \hat{\mathbf{e}}^{(k+1)} \|}{\| \hat{\mathbf{e}}^{(k)} \|} \right)}{\log \left( \frac{\| \hat{\mathbf{e}}^{(k)} \|}{\| \hat{\mathbf{e}}^{(k-1)} \|} \right)} \quad \text{for } k \text{ sufficiently large,} \quad (42)$$

### 7.3 Exercises

The text of the exercises is for Matlab users. Nonetheless, you can use Python instead.

**Exercise 7.1** (Forcing Term Functions). Define three function handles `fterms.lin`, `fterms.suplin`, `fterms.quad` such that, given  $k$  and  $\nabla f(\mathbf{x}_k)$  (as input arguments  $\mathbf{k}$  and `gradfk`), returns the linear, superlinear, and quadratic forcing terms  $\eta_k$ , respectively, described in the appendix of this document<sup>3</sup>.

Then, save this three variables in a file named `forcing_terms.mat`.

<sup>3</sup>Actually, the input  $k$  is not necessary to define the linear, superlinear, quadratic forcing terms in the appendix; nonetheless, we introduce this variable because in the next exercise, one may want to use a forcing term depending also from  $k$  and not only  $\nabla f(\mathbf{x}_k)$ .

**Exercise 7.2** (Implementation of the Inexact Newton Method). Write a Matlab function called `innnewton_bcktrck.m` that implements the inexact Newton method. Do it by modifying the `newton_bcktrck.m` function, in particular by adding the following extra input arguments:

- **`fterms`**: a function handle characterizing the sequence of forcing terms  $\eta_k$  of the inexact Newton method. This function handle must be like the ones defined in the previous exercise; i.e., depending on the input arguments **`k`** and **`gradfk`**.
- **`pcg_maxit`**: the maximum number of iterations allowed for the pcg solver to compute  $\mathbf{p}_{IN}^{(k)}$  of the inexact Newton method.

Moreover, add also the following extra output:

- **`pcg_iterseq`**: the number of pcg-iterations executed at each main iteration for computing the direction.

Once you have written the function, test it using the function handles in `forcing_terms.mat` and the data in `test_functions2.mat` (just for checking the code).

**Exercise 7.3** (Rates of Convergence). Write two functions, `expconv_xk.m` and `expconv_ek.m`, that compute a sequence of experimental orders of convergence  $\{q^{(k)}\}_{k \geq 2}$  using (42), given the sequence  $\{\mathbf{x}^{(k)}\}_{k \geq 0}$  (function `expconv_xk.m`) or the sequence  $\{\hat{\mathbf{e}}^{(k)}\}_{k \geq 1}$  (function `expconv_ek.m`).

Now, write another function, `trueconv_xk.m`, that computes a sequence of orders of convergence  $\{q^{(k)}\}_{k \geq 2}$  using (41), given the sequence  $\{\mathbf{x}^{(k)}\}_{k \geq 0}$  and the true solution  $\mathbf{x}^*$  of the minimization problem.

**Exercise 7.4** (Application to the  $n$ -dimensional Rosenbrock Function). Write a Matlab script where, with respect to the 100-dimensional, parametric, Rosenbrock function (see (103)), you compare the results obtained with the following methods:

- Steepest descent;
- Newton;
- Inexact Newton, “linear” forcing term;
- Inexact Newton, “superlinear” forcing term;
- Inexact Newton, “quadratic” forcing term;

In particular, analyze the behavior of the methods and estimate and verify their order of convergence, varying the parameter  $\alpha = 1, 5, 10, 100$  of the Rosenbrock function. How does the convergence of the methods changes, varying  $\alpha$ ?

Concerning the other parameters: **`alpha0`** =  $10^{-3}$  (steepest descent), **`x`**<sup>(0)</sup> =  $\mathbf{0} = (0, \dots, 0) \in \mathbb{R}^{100}$ , **`tolgrad`** =  $10^{-8}$ , **`c1`** =  $10^{-4}$ , **`rho`** = 0.8, **`kmax`** = 5000, **`btmax`** = 50, **`pcg_maxit`** = 50.



## 8 Newton and Inexact Newton Methods for Nonlinear Equations

Let us consider a nonlinear function  $\mathbf{F} : \mathbb{R}^n \rightarrow \mathbb{R}^n$ , with Jacobian matrix in  $\mathbb{R}^{n \times n}$  denoted by  $J\mathbf{F}$ , and a Newton-Tangent<sup>4</sup> method to solve the nonlinear system  $\mathbf{F}(\mathbf{x}) = \mathbf{0}$ , i.e.:

$$\begin{cases} \mathbf{x}^{(0)} \in \mathbb{R}^n \text{ given} \\ \mathbf{x}^{(k+1)} = \mathbf{x}^{(k)} + \mathbf{p}^{(k)}, \forall k \geq 0 \end{cases} \quad (43)$$

where  $\mathbf{p}^{(k)}$  is solution of the linear system

$$J\mathbf{F}(\mathbf{x}^{(k)}) \cdot \mathbf{p} = -\mathbf{F}(\mathbf{x}^{(k)}) \quad (44)$$

**Attention:** under suitable assumptions, the Newton-Tangent method displays a fast rate of convergence (quadratic) when it is close to a solution, but it suffers a lot from the choice of the initial guess  $\mathbf{x}^{(0)}$ ; then, if a good initial guess is not used, the method is likely to be not able to converge. Moreover, not necessarily the linear system (44) has a solution (i.e.,  $J\mathbf{F}(\mathbf{x}^{(k)})$  is singular). See Section 8.1 for more details about this method.

### 8.1 Newton-Tangent Method and Optimization of the Merit Function

Let  $f(\mathbf{x})$  be a *merit function* of  $\mathbf{F}$  such that:

$$f(\mathbf{x}) = \frac{1}{2} \|\mathbf{F}(\mathbf{x})\|^2. \quad (45)$$

We observe the following relationships between  $\mathbf{F}$  and  $f$ , especially with respect to the sequence of the Newton-Tangent method. In particular, it holds

$$\nabla f(\mathbf{x}) = J\mathbf{F}(\mathbf{x})^T \mathbf{F}(\mathbf{x}), \quad (46)$$

then:

1.  $\mathbf{F}(\mathbf{x}^*) = \mathbf{0} \Rightarrow \nabla f(\mathbf{x}^*) = \mathbf{0}$  and  $f(\mathbf{x}^*) = \mathbf{0}$  (i.e.,  $\mathbf{x}^*$  global minimizer of  $f$ );
2. ... but  $\nabla f(\hat{\mathbf{x}}) = \mathbf{0}$  (i.e.,  $\hat{\mathbf{x}}$  local minimizer<sup>5</sup> of  $f$ )  $\nRightarrow \mathbf{F}(\hat{\mathbf{x}}) = \mathbf{0}$ ; i.e., only the global minima of  $f$  are solutions of  $\mathbf{F}(\mathbf{x}) = \mathbf{0}$ , while the local minima are not;
3.  $\mathbf{p}^{(k)}$  is a descent direction for  $f$  at  $\mathbf{x}^{(k)}$  (if  $\mathbf{F}(\mathbf{x}^{(k)}) \neq \mathbf{0}$ ), because:

$$\begin{aligned} \mathbf{p}^{(k)T} \nabla f(\mathbf{x}^{(k)}) &= \mathbf{p}^{(k)T} J\mathbf{F}(\mathbf{x}^{(k)})^T \mathbf{F}(\mathbf{x}^{(k)}) = \left( J\mathbf{F}(\mathbf{x}^{(k)}) \mathbf{p}^{(k)} \right)^T \mathbf{F}(\mathbf{x}^{(k)}) = \\ &= -\mathbf{F}(\mathbf{x}^{(k)})^T \mathbf{F}(\mathbf{x}^{(k)}) = -\|\mathbf{F}(\mathbf{x}^{(k)})\|^2 < \\ &< 0. \end{aligned} \quad (47)$$

Since  $\mathbf{p}^{(k)}$  is a descent direction for  $f$  at  $\mathbf{x}^{(k)}$ , the Newton sequence (43)-(44) is also a descent method for the merit function  $f$ . Therefore, we can implement a *line search* strategy to improve the efficiency of the method. Specifically, (43)-(44) changes into

$$\begin{cases} \mathbf{x}^{(0)} \in \mathbb{R}^n \text{ given} \\ \mathbf{x}^{(k+1)} = \mathbf{x}^{(k)} + \alpha^{(k)} \mathbf{p}^{(k)}, \forall k \geq 0 \end{cases} \quad (48)$$

<sup>4</sup>we use the name “Newton-Tangent” only to distinguish this method from its homonym for optimization.

<sup>5</sup>We are not considering the case of other critical points, such as maximizers or saddle points.

where  $\mathbf{p}^{(k)}$  is given by (44) and  $\alpha^{(k)}$  is selected throughout a line search method applied with respect to the merit function  $f$ . For example, if we apply a *backtracking* strategy, we check the Armijo condition (30) with respect to the merit function  $f$ .

**Suggestion (useful for implementing the Armijo condition):** Look carefully at item 3 for a “smart” implementation of the Armijo condition.

## 8.2 Inexact Newton-Tangent Method for Nonlinear Equations

Sometimes the computation of the Newton-Tangent direction  $\mathbf{p}_N^{(k)}$ , solving the linear system (44), can be quite expensive. Therefore, in order to save computational efforts, instead of computing  $\mathbf{p}_N^{(k)}$ , we compute an “approximated Newton-Tangent direction”  $\mathbf{p}_{IN}^{(k)} \approx \mathbf{p}_N^{(k)}$ .

### 8.2.1 Inexact Descent Direction and Forcing Terms

Assuming the usage of an iterative method to solve (44), at the  $k$ -th step the iterative solver returns<sup>6</sup> a vector  $\mathbf{p}^{(k)*}$  with residual such that

$$\underbrace{\|J\mathbf{F}(\mathbf{x}^{(k)})\mathbf{p}^{(k)*} + \mathbf{F}(\mathbf{x}^{(k)})\|}_{\text{residual}} \leq \varepsilon, \quad (49)$$

where  $\varepsilon > 0$  is the arbitrary small tolerance used for the iterative method.

Then, instead of using a small and fixed  $\varepsilon$ , the Inexact Newton consists of using an adaptive tolerance

$$\varepsilon_k := \underbrace{\eta_k}_{\text{forcing terms}} \|\mathbf{F}(\mathbf{x}^{(k)})\| \quad (50)$$

that is relatively “large” during the first optimization steps; i.e., we use a large tolerance when we assume to be far from the optimum.

In general, under local and regularity assumptions, the most used forcing term sequences are the following:

- $\eta_k = 0.5$  fixed (*linear* convergence);
- $\eta_k = \min(0.5, \sqrt{\|\mathbf{F}(\mathbf{x}^{(k)})\|}) \xrightarrow{k \rightarrow \infty} 0$  (*superlinear* convergence);
- $\eta_k = \min(0.5, \|\mathbf{F}(\mathbf{x}^{(k)})\|) \in \mathcal{O}(\|\mathbf{F}(\mathbf{x}^{(k)})\|)$  (*quadratic* convergence).

**N.B.:** the rate of convergence depends also on the iterative solver; e.g. some exceptions are observable in the case where the solver, despite the larger tolerance, returns always/often a solution  $\mathbf{p}_{IN}^{(k)} \approx \mathbf{p}_N^{(k)}$  (i.e. with residual  $\sim \varepsilon$ ).

**ATTENTION:** The *gmres* Matlab function uses the *relative residual* for its stopping criteria and not the *residual* (see Section 3). Then, pay attention in defining the tolerance!

**ATTENTION:** item 3 cannot be exploited anymore for a “smart” implementation of the Armijo condition, because we have (almost always)  $\mathbf{p}_{IN}^{(k)} \neq \mathbf{p}_N^{(k)}$

## 8.3 Test Functions

For this laboratory we use as test function the functions listed in Appendix C.1.

---

<sup>6</sup>if the maximum iterations have not been reached.

## 8.4 Exercises

The text of the exercises is for Matlab users. Nonetheless, you can use Python instead.

**Exercise 8.1** (Newton Method for Nonlinear Systems). Write a Matlab function `newtonsolve_bcktrck.m` that implements the Newton method for nonlinear systems, i.e., systems  $\mathbf{F}(\mathbf{x}) = \mathbf{0}$  of  $n$  equations and  $n$  variables.

Moreover, implement the backtracking strategy, reading this method as a descent method for the merit function  $f(\mathbf{x}) = (1/2) \|\mathbf{F}(\mathbf{x})\|^2$ .

In particular, use the following inputs.

- **x0**: a column vector of  $n$  elements representing the starting point for the method;
- **F**: a function handle variable that, for each column vector  $\mathbf{x} \in \mathbb{R}^n$ , returns the vector of values  $\mathbf{F}(\mathbf{x}) \in \mathbb{R}^n$ ;
- **JF**: a function handle variable that, for each column vector  $\mathbf{x} \in \mathbb{R}^n$ , returns the value of the Jacobian matrix  $J_{\mathbf{F}}(\mathbf{x}) \in \mathbb{R}^{n \times n}$ ;
- **kmax**: an integer scalar value characterizing the maximum number of iterations of the method;
- **Ftol**: a real scalar value defining the tolerance with respect to the norm of  $\mathbf{F}$  to stop the method when we are near to the solution of the system.
- **c1**: the factor  $c_1$  for the Armijo condition that must be a scalar in  $(0, 1)$ ;
- **rho**: fixed factor, less than 1, used to reduce the step factor during the backtracking strategy;
- **btmax**: maximum number of steps of the backtracking strategy.

The outputs of the function must be

- **xk**: the last vector  $\mathbf{x}^{(k)} \in \mathbb{R}^n$  computed by the method before it stops;
- **normFk**: the value  $\|\mathbf{F}(\mathbf{x}^{(k)})\|$ ;
- **k**: index value of the last step executed by the optimization method before stopping;
- **xseq**: a matrix/vector in  $\mathbb{R}^{n \times k}$  such that each column  $j$  is the vector  $\mathbf{x}^{(j)} \in \mathbb{R}^n$  generated by the iterations of the method.
- **btseq**: row vector in  $\mathbb{R}^k$  such that the  $j$ -th element is the number of backtracking iterations done at the  $j$ -th step of the method.

Once you have written the function, test it using the data inside the file `test_nonlinsys.mat`.

**Exercise 8.2** (Implementation of the Inexact Newton-Tangent Method). Write a Matlab function, `innewtonsolve_bcktrck.m`, that implements the inexact Newton-Tangent method. Do it modifying the `newtonsolve_bcktrck.m` function, in particular adding the following extra inputs:

- **fterms**: a function handle characterizing the sequence of forcing terms  $\eta_k$  of the inexact Newton method. This function handle must be like the ones defined in the previous exercise; i.e., depending on the input arguments **k** and **gradfk**.
- **gmres\_maxit\_inner** and **gmres\_maxit\_outer**: the maximum number of inner/outer iterations allowed for the gmres solver to compute  $\mathbf{p}_{IN}^{(k)}$  of the inexact Newton-Tangent method.

Moreover, add also the following extra output:

- **gmresiterseq**: the number of inner/outer gmres-iterations executed at each main iteration for computing the direction.

Once you have written the function, test it using the function handles in `test_nonlinsys.mat` (just for checking the code).

**Exercise 8.3** (Applications). Given the function of the previous exercise, write a Matlab script where you test these functions on the data inside the file `test_nonlinsys.mat` (containing the test functions of Section C.1). Try both these starting points: with  $\mathbf{x}^{(0)} = (-5, -2.5), (-5, -5) \in \mathbb{R}^2$ . For the backtracking parameters, use the same values as the exercises in previous sections.

Then, given  $\mathbf{F} = (f_1, f_2)$  and  $\mathbf{G} = (g_1, g_2)$ , plot:

- the **contour** and the **surf** plots of the functions  $f_1, f_2$  and the sequence `xseq`. Repeat also for  $g_1, g_2$ ;
- try to merge the previous plots, observing the different path of the sequence `xseq` on both  $f_1$  and  $f_2$ . Repeat for  $g_1, g_2$ .
- using the subplot command, observe “in parallel” the path of the sequence `xseq` both in the  $(x_1, x_2)$ -plane (the domain) and in the  $(f_1, f_2)$ -plane (the codomain). Repeat for  $g_1, g_2$ .

**Special request:** try also to implement the case where the Jacobian is unknown and must be approximated.

## 9 Gauss-Newton for Nonlinear Least Squares

Let us consider a nonlinear function  $\mathbf{F} : \mathbb{R}^n \rightarrow \mathbb{R}^m$ ,  $\mathbf{F} = (f_1, \dots, f_m)$ ,  $m \geq n$ , with Jacobian matrix in  $\mathbb{R}^{m \times n}$  denoted by  $J\mathbf{F}$ , and a Gauss-Newton method for finding a vector  $\mathbf{x}^*$  solution of the nonlinear least square problem

$$\min_{\mathbf{x} \in \mathbb{R}^n} f(\mathbf{x}), \quad (51)$$

where  $f : \mathbb{R}^n \rightarrow \mathbb{R}_{\geq 0}$  is the merit function

$$f(\mathbf{x}) = \frac{1}{2} \|\mathbf{F}(\mathbf{x})\|^2. \quad (52)$$

In particular, the Gauss-Newton method looks for a solution of (51) building a sequence  $\{\mathbf{x}^{(k)}\}_{k \in \mathbb{N}}$  such that:

$$\begin{cases} \mathbf{x}^{(0)} \in \mathbb{R}^n \text{ given} \\ \mathbf{x}^{(k+1)} = \mathbf{x}^{(k)} + \mathbf{p}^{(k)}, \forall k \geq 0 \end{cases} \quad (53)$$

where  $\mathbf{p}^{(k)}$  is solution of the linear system

$$\underbrace{J\mathbf{F}(\mathbf{x}^{(k)})^T J\mathbf{F}(\mathbf{x}^{(k)})}_{\approx Hf(\mathbf{x}^{(k)})} \mathbf{p} = - \underbrace{J\mathbf{F}(\mathbf{x}^{(k)})^T \mathbf{F}(\mathbf{x}^{(k)})}_{=\nabla f(\mathbf{x}^{(k)})}, \quad (54)$$

where the matrix of the system is an approximation of the Hessian of the merit function, because:

$$Hf(\mathbf{x}^{(k)}) = J\mathbf{F}(\mathbf{x}^{(k)})^T J\mathbf{F}(\mathbf{x}^{(k)}) + \sum_{j=1}^m f_j(\mathbf{x}^{(k)}) \cdot Hf_j(\mathbf{x}^{(k)}). \quad (55)$$

**Observation:**  $\mathbf{p}^{(k)}$  is a descent direction at  $\mathbf{x}^{(k)}$  for  $f$  if  $J\mathbf{F}(\mathbf{x}^{(k)})$  is full-rank, because:

$$\begin{aligned} \nabla f^T(\mathbf{x}^{(k)}) \mathbf{p}^{(k)} &= (J\mathbf{F}(\mathbf{x}^{(k)})^T \mathbf{F}(\mathbf{x}^{(k)}))^T \mathbf{p}^{(k)} = \\ &= -(J\mathbf{F}(\mathbf{x}^{(k)})^T J\mathbf{F}(\mathbf{x}^{(k)})) \mathbf{p}^{(k)} = \\ &= -\mathbf{p}^{(k)T} \underbrace{(J\mathbf{F}(\mathbf{x}^{(k)})^T J\mathbf{F}(\mathbf{x}^{(k)}))}_{\text{PD for full-rank of Jac.}} \mathbf{p}^{(k)} < 0 \end{aligned} \quad (56)$$

**Observation:** from (56), we have

$$\nabla f^T(\mathbf{x}^{(k)}) \mathbf{p}^{(k)} = -(\mathbf{p}^{(k)T} (J\mathbf{F}(\mathbf{x}^{(k)})^T J\mathbf{F}(\mathbf{x}^{(k)})) \mathbf{p}^{(k)}) = -\|J\mathbf{F}(\mathbf{x}^{(k)}) \mathbf{p}^{(k)}\|^2.$$

### 9.1 Gauss-Newton and Linear Least Squares

Remember that  $\mathbf{p}^{(k)}$  solution of (54) means that it is equivalently the solution of the *linear* least square problem:

$$\min_{\mathbf{p} \in \mathbb{R}^n} \frac{1}{2} \|J\mathbf{F}(\mathbf{x}^{(k)}) \mathbf{p} + \mathbf{F}(\mathbf{x}^{(k)})\|^2, \quad (57)$$

where the matrix  $A$  is  $J\mathbf{F}(\mathbf{x}^{(k)})$  and the vector  $\mathbf{b}$  is  $-\mathbf{F}(\mathbf{x}^{(k)})$ .

**Remember:** in linear least squares,  $\mathbf{F}(\mathbf{x}) = A\mathbf{x} - \mathbf{b}$ . Then, the gradient of the merit function is  $A^T A\mathbf{x} - A^T \mathbf{b}$  and the solution is obtained solving the linear system

$$A^T A\mathbf{x} = A^T \mathbf{b}.$$

**Observation:** the Gauss-Newton method can be seen both as: *i*) a second-order method for minimizing the nonlinear merit function, with an approximated Hessian for reducing computational costs (see (54) and (55)); *ii*) a local linearization of the nonlinear system, then the application of the linear least squares (see (57)) for minimizing the nonlinear merit function.

## 9.2 Least Squares and Regression in Machine Learning

Least squares problems are hidden in many Machine Learning (ML) methods; then, we can use the linear least squares or the Gauss-Newton method for “training” ML models.

### 9.2.1 Linear Least Squares and Linear Regression

Let  $\mathcal{T} = \{(\mathbf{x}_i, \tilde{y}_i)\}_{i=1}^m \subset \mathbb{R}^{n-1} \times \mathbb{R}$  be the training set representing a “linear phenomenon”, i.e.:

$$\tilde{y}_i = \mathbf{w}^{*T} \mathbf{x}_i + b^* + \varepsilon_i,$$

where  $\varepsilon_i \in \mathcal{N}(0, 0.1)$  is a white noise and where  $\mathbf{w}^* \in \mathbb{R}^{n-1}$  and  $b^* \in \mathbb{R}$  are unknown.

We want to find the parameters  $\mathbf{w}^* \in \mathbb{R}^{n-1}$  and  $b^* \in \mathbb{R}$  that generated the data, by finding the solution of the minimization problem

$$\min_{\mathbf{w}, b} \frac{1}{2} \sum_{i=1}^m (\mathbf{w}^T \mathbf{x}_i + b - \tilde{y}_i)^2. \quad (58)$$

Let  $\text{Err} : \mathbb{R}^n \rightarrow \mathbb{R}^m$  be a function such that

$$\text{Err}(\boldsymbol{\omega}) := \text{Err}(\mathbf{w}, b) = \begin{bmatrix} \mathbf{w}^T \mathbf{x}_1 + b - \tilde{y}_1 \\ \vdots \\ \mathbf{w}^T \mathbf{x}_m + b - \tilde{y}_m \end{bmatrix} = X\boldsymbol{\omega} - \tilde{\mathbf{y}},$$

where

$$X = \begin{bmatrix} \mathbf{x}_1^T & | & 1 \\ \vdots & | & \vdots \\ \mathbf{x}_m^T & | & 1 \end{bmatrix} \in \mathbb{R}^{m \times n} \quad \text{and} \quad \tilde{\mathbf{y}} = \begin{bmatrix} \tilde{y}_1 \\ \vdots \\ \tilde{y}_m \end{bmatrix} \in \mathbb{R}^m$$

Therefore, (58) can be rewritten as

$$\min_{\mathbf{w}, b} \frac{1}{2} \|\text{Err}(\mathbf{w}, b)\|^2 = \min_{\boldsymbol{\omega}} \frac{1}{2} \|X\boldsymbol{\omega} - \tilde{\mathbf{y}}\|^2, \quad (59)$$

That is the merit function of the linear least square problem with matrix  $A \rightsquigarrow X \in \mathbb{R}^{m \times n}$ , unknowns  $\mathbf{x} \rightsquigarrow \boldsymbol{\omega} \in \mathbb{R}^n$  and vector  $\mathbf{b} \rightsquigarrow \tilde{\mathbf{y}} \in \mathbb{R}^m$ .

### 9.2.2 Gauss-Newton and Nonlinear Regression

We can generalize the content of Section 9.2 to the case of nonlinear regression.

Let  $\mathcal{T} = \{(\mathbf{x}_i, \tilde{y}_i)\}_{i=1}^m \subset \mathbb{R}^k \times \mathbb{R}$  be the training set representing a “nonlinear phenomenon”, i.e.:

$$\tilde{y}_i = F^*(\mathbf{x}_i) + \varepsilon_i,$$

where  $\varepsilon_i \in \mathcal{N}(0, 0.1)$  is a white noise and  $F^* = F(\cdot; \boldsymbol{\omega}^*) : \mathbb{R}^k \rightarrow \mathbb{R}$  is a “known” nonlinear function characterized by an unknown parameter  $\boldsymbol{\omega}^* \in \mathbb{R}^n$ .

We want to find the parameters  $\boldsymbol{\omega}^* \in \mathbb{R}^n$  that generated the data, by finding the solution of the minimization problem

$$\min_{\boldsymbol{\omega}} \frac{1}{2} \sum_{i=1}^m (F(\mathbf{x}_i; \boldsymbol{\omega}) - \tilde{y}_i)^2, \quad (60)$$

that, generalizing the observations of Section 9.2, can be rewritten as

$$\min_{\boldsymbol{\omega}} \frac{1}{2} \|\text{Err}(\boldsymbol{\omega})\|^2, \quad (61)$$

where  $\text{Err} : \mathbb{R}^n \rightarrow \mathbb{R}^m$  is the function

$$\text{Err}(\boldsymbol{\omega}) := \begin{bmatrix} F(\mathbf{x}_1; \boldsymbol{\omega}) - \tilde{y}_1 \\ \vdots \\ F(\mathbf{x}_m; \boldsymbol{\omega}) - \tilde{y}_m \end{bmatrix}.$$

Therefore, we can easily observe that (61) is the minimization of the merit function of a nonlinear least square problem based on  $\text{Err}$ .

### 9.3 Test Functions

For this laboratory we use as test function the functions in `test_leastsq.mat`, that are:

- $A\mathbf{x} - \mathbf{b}$  such that

$$A = \begin{bmatrix} 1 & 1 \\ 0.9 & 1 \\ 1.1 & 1 \end{bmatrix}, \quad \mathbf{b} = \begin{bmatrix} 0 \\ -0.2 \\ 0.4 \end{bmatrix} \quad (62)$$

- $\mathbf{F} : \mathbb{R}^2 \rightarrow \mathbb{R}^3$  such that

$$\mathbf{F}(\mathbf{x}) = \begin{bmatrix} f_1(\mathbf{x}) \\ f_2(\mathbf{x}) \\ f_3(\mathbf{x}) \end{bmatrix} = \begin{bmatrix} x_1^2 + x_2^2 - 4 \\ -x_1^2 + x_2 \\ x_2 - 3 \end{bmatrix}, \quad (63)$$

and with jacobian

$$J\mathbf{F}(\mathbf{x}) = \begin{bmatrix} 2x_1 & 2x_2 \\ -2x_1 & 1 \\ 0 & 1 \end{bmatrix}. \quad (64)$$

### 9.4 Exercises

The text of the exercises is for Matlab users. Nonetheless, you can use Python instead.

**Exercise 9.1** (Linear Least Squares). *Write a Matlab function `lin_leastsq.m` that computes the solution for a given linear least square problem*

$$\min_{\mathbf{x} \in \mathbb{R}^n} \frac{1}{2} \|A\mathbf{x} - \mathbf{b}\|^2, \quad (65)$$

where  $A \in \mathbb{R}^{m \times n}$ ,  $\mathbf{b} \in \mathbb{R}^m$ ,  $m \geq n$ .

In particular, use the following inputs.

- $A$ : the matrix  $A \in \mathbb{R}^{m \times n}$ ,  $m \geq n$ , of the least square problem;
- $\mathbf{b}$ : the vector  $\mathbf{b} \in \mathbb{R}^m$  of the least square problem.

The outputs of the function must be

- $\mathbf{x}$ : the vector  $\mathbf{x}^* \in \mathbb{R}^n$  computed solving the least square problem;
- $F\mathbf{x}$ : the vector  $A\mathbf{x}^* - \mathbf{b}$ ;
- $\text{normF}\mathbf{x}$ : the value  $\|A\mathbf{x}^* - \mathbf{b}\|$ ;

- **normgradf $\mathbf{x}$** : the value  $\|\nabla f(\mathbf{x}_k)\|$ ;
- **Afullrank**: boolean value telling us if  $A$  is a full-rank matrix.

Once you have written the function, test it using the data inside the file `test_leaststsq.mat`.

**Exercise 9.2** (Gauss-Newton with Backtracking). Write a Matlab function `gaussnewton.m` that computes the solution of a given least squares problem as (51) using the Gauss-Newton method (with backtracking).

In particular, use the following inputs.

- **x0**: a column vector of  $n$  elements representing the starting point for the method;
- **F**: a function handle variable that, for each column vector  $\mathbf{x} \in \mathbb{R}^n$ , returns the vector of values  $\mathbf{F}(\mathbf{x}) \in \mathbb{R}^m$ ,  $m \geq n$ ;
- **JF**: a function handle variable that, for each column vector  $\mathbf{x} \in \mathbb{R}^n$ , returns the value of the Jacobian matrix  $\mathbf{JF}(\mathbf{x}) \in \mathbb{R}^{m \times n}$ ,  $m \geq n$ ;
- **kmax**: an integer scalar value characterizing the maximum number of iterations of the method;
- **gradftol**: a real scalar value defining the tolerance with respect to the norm of  $\nabla f$  to stop the method when we are near to the solution of the problem.
- **c1**: the factor  $c_1$  for the Armijo condition that must be a scalar in  $(0, 1)$ ;
- **rho**: fixed factor, less than 1, used to reduce the step factor during the backtracking strategy;
- **btmax**: maximum number of steps of the backtracking strategy.

The outputs of the function must be

- **xk**: the last vector  $\mathbf{x}^{(k)} \in \mathbb{R}^n$  computed by the method before it stops;
- **Fx**: the vector  $\mathbf{F}(\mathbf{x}^{(k)})$ ;
- **normFk**: the value  $\|\mathbf{F}(\mathbf{x}^{(k)})\|$ ;
- **normgradfk**: the value  $\|\nabla f(\mathbf{x}^{(k)})\|$ ;
- **k**: index value of the last step executed by the optimization method before stopping;
- **xseq**: a matrix/vector in  $\mathbb{R}^{n \times k}$  such that each column  $j$  is the vector  $\mathbf{x}^{(j)} \in \mathbb{R}^n$  generated by the iterations of the method.
- **btseq**: row vector in  $\mathbb{R}^k$  such that the  $j$ -th element is the number of backtracking iterations done at the  $j$ -th step of the method.

Once you have written the function, test it using the data inside the file `test_leaststsq.mat`.

**Exercise 9.3** (Applications). Given the function of the previous exercises, write a Matlab script where you test these functions using the data inside the file `test_leaststsq.mat` (containing the test functions of Section 9.3). Try the starting points  $\mathbf{x}^{(0)} = (-0.5, -0.5) \in \mathbb{R}^2$  for Gauss-Newton. For the backtracking parameters, use the same values of the exercises in previous sections.

Then, given  $\mathbf{Ax} - \mathbf{b}$  and  $\mathbf{F} = (f_1, f_2, f_3)$ , plot:

- the **contour** and the **surf** plots of the system functions all together and the sequence **xseq** (for Gauss-Newton, otherwise the solution).



**Special request:** try also to implement the case where the Jacobian is unknown and must be approximated.

**Exercise 9.4** (Least Squares and Regression). Given the function of the previous exercises, write a Matlab script where you use them to find the parameters characterizing a line

$$\ell(x) = wx + b,$$

a curve

$$\phi(x) = w(g(x)) + b, \quad \text{where } g(x) := 1.25 \sin(0.45x^2 + 0.13),$$

and a curve

$$h(x) = \alpha \sin(\varphi x + \psi),$$

given the noisy data inside `leastsq_regression.mat`.

In particular, in both the datasets, we have pairs  $(x_i, \tilde{y}_i)$  with  $\tilde{y}_i = \ell(x_i) + \varepsilon_i$ ,  $\tilde{y}_i = \phi(x_i) + \varepsilon_i$  and  $\tilde{y}_i = h(x_i) + \varepsilon_i$ , respectively, where  $\varepsilon_i \in \mathcal{N}(0, 0.1)$  is a white noise.

Then, use the linear least square matlab function of the first exercise to learn an approximation of  $w$  and  $b$  from the data, both for  $\ell$  and  $\phi$ .

Analogously, use the Gauss-Newton matlab function of the second exercise to learn an approximation of  $\alpha$ ,  $\varphi$  and  $\psi$  from the data.

**Suggestion:** now  $(x_i, \tilde{y}_i)$  are parameters, while  $(w, b)$  and  $(\alpha, \varphi, \psi)$  are the variables of the least square problems. Then:

- Linear case -  $\ell$ :

$$A \rightsquigarrow X := \begin{bmatrix} x_1 & 1 \\ \vdots & \vdots \\ x_m & 1 \end{bmatrix} \in \mathbb{R}^{m \times 2}, \quad \mathbf{b} \rightsquigarrow \tilde{\mathbf{y}} := \begin{bmatrix} \tilde{y}_1 \\ \vdots \\ \tilde{y}_m \end{bmatrix} \in \mathbb{R}^m, \quad \mathbf{x} \rightsquigarrow (w, b) \in \mathbb{R}^2$$

- “Fake” Nonlinear case -  $\phi$ :

$$A \rightsquigarrow X := \begin{bmatrix} g(x_1) & 1 \\ \vdots & \vdots \\ g(x_m) & 1 \end{bmatrix} \in \mathbb{R}^{m \times 2}, \quad \mathbf{b} \rightsquigarrow \tilde{\mathbf{y}} := \begin{bmatrix} \tilde{y}_1 \\ \vdots \\ \tilde{y}_m \end{bmatrix} \in \mathbb{R}^m, \quad \mathbf{x} \rightsquigarrow (w, b) \in \mathbb{R}^2$$

- Nonlinear case -  $h$ :

$$\mathbf{F}(\mathbf{x}) \rightsquigarrow \mathbf{F}(\alpha, \varphi, \psi) = \begin{bmatrix} \alpha \sin(\varphi x_1 + \psi) - \tilde{y}_1 \\ \vdots \\ \alpha \sin(\varphi x_m + \psi) - \tilde{y}_m \end{bmatrix}, \quad J\mathbf{F}(\mathbf{x}) \rightsquigarrow \text{exercise}, \quad \mathbf{x} \rightsquigarrow (\alpha, \varphi, \psi) \in \mathbb{R}^3.$$

## 10 Projected Gradient

Let  $f : \mathbb{R}^n \rightarrow \mathbb{R}$  be a loss function that have to be minimized with respect to a closed convex set  $X \subset \mathbb{R}^n$  of constrains, i.e. we want to find

$$\arg \min_{\mathbf{x} \in X} f(\mathbf{x}).$$

The *Projected Gradient* (PG) method is characterized by the following main steps (at the  $k$ -th iteration):

1. compute the steepest descent direction  $\mathbf{p}^{(k)} = -\nabla f(\mathbf{x}^{(k)})$ ;
2. given an arbitrary sequence of  $\gamma^{(k)} > 0$ , we compute  $\hat{\mathbf{x}}^{(k)} = (\mathbf{x}^{(k)} + \gamma^{(k)} \mathbf{p}^{(k)})$  and its projection  $\tilde{\mathbf{x}}^{(k)} = \Pi_X(\hat{\mathbf{x}}^{(k)})$  onto  $X$ . Remember,  $\tilde{\mathbf{x}}^{(k)} \equiv \hat{\mathbf{x}}^{(k)}$  if  $\hat{\mathbf{x}}^{(k)} \in X$ .
3. compute the *feasible* direction  $\boldsymbol{\pi}^{(k)} := (\tilde{\mathbf{x}}^{(k)} - \mathbf{x}^{(k)})$ ;
4. compute the next step of the optimization method:  $\mathbf{x}^{(k+1)} = \mathbf{x}^{(k)} + \alpha^{(k)} \boldsymbol{\pi}^{(k)}$ , where  $\alpha_k \in (0, 1]$  is the sequence of step length factors.

Then, we can summarize the PG method as

$$\begin{cases} \mathbf{x}^{(0)} \in X \text{ given} \\ \mathbf{x}^{(k+1)} = \mathbf{x}^{(k)} + \alpha^{(k)} \boldsymbol{\pi}^{(k)}, \forall k \geq 0 \end{cases} \quad (66)$$

### 10.1 Convergence Properties to Stationary Points

In the case of constrained optimization in convex domains, we can extend the convergence results for the unconstrained optimization (under proper assumptions) to a *stationary point*  $\mathbf{x}^*$ ; i.e., a point such that:

$$\mathbf{x}^* \in X \text{ is a stationary point} \iff \nabla f(\mathbf{x}^*)^T (\mathbf{x} - \mathbf{x}^*) \geq 0, \quad \forall \mathbf{x} \in X. \quad (67)$$

**Proposition 10.1** (Stationary Points Characterization).  $\mathbf{x}^* \in X$  is a stationary point for  $f$  in  $X$  if and only if

$$\Pi_X(\underbrace{\mathbf{x}^* - \gamma \nabla f(\mathbf{x}^*)}_{=\hat{\mathbf{x}}^*}) = \mathbf{x}^*, \quad \forall \gamma > 0. \quad (68)$$

**Attention:** Due to the objective of converging toward stationary points, the minimum not necessarily corresponds to a point with null gradient. Therefore, the implementation of the PG can use (68) as a stopping criterium, or can use two stopping criteria together: one with respect to the norm of the gradient and one with respect to the length of the step  $\|\mathbf{x}^{(k+1)} - \mathbf{x}^{(k)}\|$ .

### 10.2 Projected Gradient and Line Search

In a nutshell, two main line search methods exist for the Projected Gradient method:

1. **Line Search along the feasible direction  $\boldsymbol{\pi}^{(k)}$ :** we assume a constant sequence  $\gamma^{(k)} \equiv \gamma > 0$ , for each  $k \in \mathbb{N}$ , and we perform a line search method with respect to  $\alpha_k$  along the direction  $\boldsymbol{\pi}^{(k)}$  (starting from the value  $\alpha_0^{(k)} = 1$ ).
2. **Linea Search along the projection arc:** we assume the sequence  $\alpha^{(k)}$  constantly equal to 1; then, by construction, we have that

$$\begin{aligned} \mathbf{x}^{(k+1)} &= \mathbf{x}^{(k)} + \boldsymbol{\pi}^{(k)} = \tilde{\mathbf{x}}^{(k)} = \Pi_X(\hat{\mathbf{x}}^{(k)}) = \\ &= \Pi_X(\mathbf{x}^{(k)} + \gamma^{(k)} \mathbf{p}^{(k)}) = \\ &= \Pi_X(\mathbf{x}^{(k)} - \gamma^{(k)} \nabla f(\mathbf{x}^{(k)})), \end{aligned} \quad (69)$$

and we can perform a line search method with respect to the parameter  $\gamma^{(k)}$  along the projection arc.

### 10.3 Projection Functions

Here, some example of function  $\Pi_X$  with  $X$  as a sphere or a box in  $\mathbb{R}^n$ .

#### 10.3.1 $n$ -dimensional Sphere

Let  $\mathbf{c} = (c_1, \dots, c_n) \in \mathbb{R}^n$  be the center of the sphere  $X$  and let  $r \in \mathbb{R}^+$  be its radius. Then, it holds

$$\Pi_X(\mathbf{y}) = \begin{cases} \mathbf{y}, & \text{if } \|\mathbf{y} - \mathbf{c}\| \leq r \\ \mathbf{c} + r \frac{\mathbf{y} - \mathbf{c}}{\|\mathbf{y} - \mathbf{c}\|}, & \text{otherwise} \end{cases}. \quad (70)$$

#### 10.3.2 $n$ -dimensional Box

Let  $X$  be the  $n$ -dimensional box  $[m_1, M_1] \times \dots \times [m_n, M_n] \subset \mathbb{R}^n$ . Then, it holds

$$\Pi_X(\mathbf{y}) = \begin{bmatrix} \Pi_{[m_1, M_1]}(y_1) \\ \vdots \\ \Pi_{[m_n, M_n]}(y_n) \end{bmatrix} \in \mathbb{R}^n \quad (71)$$

where

$$\Pi_{[m_i, M_i]}(y_i) = \begin{cases} m_i, & \text{if } y_i < m_i \\ M_i, & \text{if } y_i > M_i \\ y_i, & \text{otherwise} \end{cases}. \quad (72)$$

### 10.4 Exercises

The text of the exercises is for Matlab users. Nonetheless, you can use Python instead.

**Exercise 10.1** (Projections). Write two Matlab functions `sphere_projection.m` and `box_projection.m` that, for each  $\mathbf{y} \in \mathbb{R}^n$ , return the projection of  $\mathbf{y}$  on a given  $n$ -dimensional sphere and on a given  $n$ -dimensional box, respectively.

In particular, the inputs of `sphere_projection.m` must be

- $\mathbf{y}$ : the input column vector  $\mathbf{y} \in \mathbb{R}^n$ ;
- $\mathbf{c}$ : the column vector  $\mathbf{c} \in \mathbb{R}^n$  representing the center of the sphere;
- $r$ : the radius  $r > 0$  of the sphere.

On the other hand, the inputs of `box_projection.m` must be

- $\mathbf{x}$ : the input column vector  $\mathbf{x} \in \mathbb{R}^n$ ;
- $\mathbf{mins}$ :  $n$ -dimensional column vector  $[m_1, \dots, m_n]^\top$ ;
- $\mathbf{maxs}$ :  $n$ -dimensional column vector  $[M_1, \dots, M_n]^\top$ .

Both the functions must have as unique output the vector  $\mathbf{x}_{hat}$ , corresponding to the projected vector.

**Exercise 10.2** (Projected Gradient Implementation). Write a Matlab function `projgrad_bcktrck.m` that implements the Projected Gradient optimization method with backtracking strategy with respect to  $\alpha_k$  (i.e., fixed  $\gamma^{(k)} \equiv \gamma > 0$ ).

In particular, start from the code of the function `steepest_desc_bcktrck.m` and modify it, removing unnecessary inputs while adding the following ones:

- **gamma**: fixed factor  $\gamma > 0$  that multiplies the descent direction before the (possible) projection on  $\partial X$ , where  $X$  is the optimization domain;
- **tolx**: a real scalar value characterizing the tolerance with respect to the norm of  $\|\mathbf{x}^{(k+1)} - \mathbf{x}^{(k)}\|$  to stop the method;
- **Pi\_X**: a function handle variable that characterizes the function  $\Pi_X$  with respect to an arbitrary set  $X \subset \mathbb{R}^n$ . For each column vector  $\mathbf{x} \in \mathbb{R}^n$ ,  $\Pi_X$  returns  $\mathbf{x}$  if  $\mathbf{x} \in X$ , and its projection on  $\partial X$  otherwise. The function handle must take only  $\mathbf{x}$  as input argument.

Due to the additional stopping criterium, add also the following output:

- **deltaxk\_norm**: the euclidean norm  $\|\mathbf{x}^{(k)} - \mathbf{x}^{(k-1)}\|$  (where  $k$  is intended to be the last iteration).
- **[Optional] statpk\_norm**: the euclidean norm  $\|\Pi_X(\mathbf{x}^{(k)} - \gamma \nabla f(\mathbf{x}^{(k)})) - \mathbf{x}^{(k)}\|$  (where  $k$  is intended to be the last iteration).

Once you have written the function, test it using the data in `test_functions2.mat`.

**Exercise 10.3** (Projected Gradient). Write a Matlab script where you test the function `projgrad_bcktrck.m` using the data inside the file `test_functions2.mat`.

Then, for each  $f = f_1, f_2, f_3$  (see Section B.1) in `test_functions2.mat`, plot:

- A top view of the loss  $f$  using the Matlab function `contour`, together with both the sequence  $\{\mathbf{x}^{(k)}\}$  in  $\mathbb{R}^2$  and the boundary of  $X$ ;
- the surface of the loss  $f$  using the Matlab function `surf`, together with the function values of both the sequence  $\{\mathbf{x}^{(k)}\}$  in  $\mathbb{R}^3$  and the boundary of  $X$ .

Concerning the other parameters:  $c1 = 10^{-4}$ ,  $\rho = 0.8$ ,  $btmax = 50$ ,  $\gamma = 0.1$ ,  $tolx = 10^{-8}$ . Concerning the sphere center, sphere radius, or box domain, generate them randomly or on your choice.

**Exercise 10.4** (Projected Gradient Implementation). Write a Matlab function `projgrad2_bcktrck.m` that implements the Projected Gradient optimization method with backtracking strategy with respect to  $\alpha_k$  (i.e., fixed  $\gamma^{(k)} \equiv \gamma > 0$ ) that is equivalent to the function described in Exercise 10.2 but that uses (68) as a stopping criterium.

## 11 Quadratic Programming with Eq. Constraints (Lab. 11)

Given a generic minimization problem

$$\begin{cases} \min_{\mathbf{x} \in \mathbb{R}^n} f(\mathbf{x}) \\ \mathbf{g}(\mathbf{x}) \geq \mathbf{0} \in \mathbb{R}^k \\ \mathbf{h}(\mathbf{x}) = \mathbf{0} \in \mathbb{R}^m \end{cases}, \quad (73)$$

where  $f : \mathbb{R}^n \rightarrow \mathbb{R}$ ,  $\mathbf{g} : \mathbb{R}^n \rightarrow \mathbb{R}^k$ , and  $\mathbf{h} : \mathbb{R}^n \rightarrow \mathbb{R}^m$ , we define the *Lagrangian function* of the problem as

$$\mathcal{L}(\mathbf{x}, \boldsymbol{\lambda}, \boldsymbol{\mu}) := f(\mathbf{x}) - \boldsymbol{\lambda}^T \mathbf{g}(\mathbf{x}) - \boldsymbol{\mu}^T \mathbf{h}(\mathbf{x}). \quad (74)$$

Then, the *Karush-Kuhn-Tucker* (KKT) conditions for the problem are the system of equalities and inequalities:

$$\begin{cases} \nabla_{\mathbf{x}} \mathcal{L}(\mathbf{x}, \boldsymbol{\lambda}, \boldsymbol{\mu}) = \mathbf{0} \\ \boldsymbol{\lambda}^T \mathbf{g}(\mathbf{x}) = 0 \\ \boldsymbol{\lambda} \geq \mathbf{0} \\ \mathbf{g}(\mathbf{x}) \geq \mathbf{0} \\ \mathbf{h}(\mathbf{x}) = \mathbf{0} \end{cases}; \quad (75)$$

moreover, we recall the important theorem that exploits the KKT conditions to identify a minimum of  $f$  that satisfies the constraints.

**Theorem 11.1** (Karush-Kuhn-Tucker Optimality Conditions). *Let  $\mathbf{x}^* \in \mathbb{R}^n$  be a local minimum for a constraint minimization problem such that it is a regular point, i.e.:*

$$\begin{aligned} \nabla_{\mathbf{x}} h_i(\mathbf{x}^*) &\text{ are lin. ind. for each } i = 1, \dots, m, \\ \nabla_{\mathbf{x}} g_j(\mathbf{x}^*) &\text{ are lin. ind. for each } j \in \mathcal{A}(\mathbf{x}^*), \end{aligned} \quad (76)$$

where  $\mathcal{A}(\mathbf{x}) := \{j \in \{1, \dots, k\} \mid g_j(\mathbf{x}) = 0\}$  (active set at  $\mathbf{x}$ ).

Then, there are  $\boldsymbol{\lambda}^* \in \mathbb{R}^k$  and  $\boldsymbol{\mu}^* \in \mathbb{R}^m$  such that  $(\mathbf{x}^*, \boldsymbol{\lambda}^*, \boldsymbol{\mu}^*)$  satisfy the KKT (see (75)).

### 11.1 Equivalent Definition of $\mathcal{L}$

The inequality and equality constraints of problem (73) can be written also as  $-\mathbf{g}(\mathbf{x}) \leq \mathbf{0}$  and  $-\mathbf{h}(\mathbf{x}) = \mathbf{0}$ , respectively, without changing the actual constraints of the problem. Then, varying the formulation of the problem, also the Lagrangian function may change its expression; i.e.:

$$\mathcal{L}(\mathbf{x}, \boldsymbol{\lambda}, \boldsymbol{\mu}) := f(\mathbf{x}) \pm \boldsymbol{\lambda}^T \mathbf{g}(\mathbf{x}) \pm \boldsymbol{\mu}^T \mathbf{h}(\mathbf{x}), \quad (77)$$

where we have  $\pm \boldsymbol{\lambda}^T \mathbf{g}(\mathbf{x})$  if the inequality constraints are  $\mp \mathbf{g}(\mathbf{x}) \leq \mathbf{0}$  and we have  $\mp \boldsymbol{\mu}^T \mathbf{h}(\mathbf{x})$  if the equality constraints are  $\pm \mathbf{h}(\mathbf{x}) = \mathbf{0}$ .

### 11.2 KKT Conditions for QP Problems

A *Quadratic Programming* (QP) problem is a minimization problem stated as

$$\begin{cases} \min_{\mathbf{x} \in \mathbb{R}^n} \frac{1}{2} \mathbf{x}^T Q \mathbf{x} + \mathbf{c}^T \mathbf{x} \\ \mathbf{g}(\mathbf{x}) = A^{(i)} \mathbf{x} - \mathbf{b}^{(i)} \geq \mathbf{0} \in \mathbb{R}^k & (k \text{ inequality constraints}) \\ \mathbf{h}(\mathbf{x}) = A^{(e)} \mathbf{x} - \mathbf{b}^{(e)} = \mathbf{0} \in \mathbb{R}^m & (m \text{ equality constraints}) \end{cases} \quad (78)$$

where  $Q \in \mathbb{R}^{n \times n}$  is a symmetric matrix; however, in this recap we focus on the *Convex* case of QP, that holds when  $Q$  is *Positive Semi-Definite*<sup>7</sup>.

<sup>7</sup>obviously, QP is *Strictly Convex* if  $Q$  is *Positive Definite*.

Then, the KKT Conditions for QP Problems are characterized by the following expressions:

$$\begin{cases} \nabla_{\mathbf{x}} \mathcal{L}(\mathbf{x}, \boldsymbol{\lambda}, \boldsymbol{\mu}) = Q\mathbf{x} + (\mathbf{c} \pm A^{(i)T} \boldsymbol{\lambda} \pm A^{(e)T} \boldsymbol{\mu}) = \mathbf{0} \\ \boldsymbol{\lambda}^T \mathbf{g}(\mathbf{x}) = \boldsymbol{\lambda}^T (A^{(i)} \mathbf{x} - \mathbf{b}^{(i)}) = 0 \\ \boldsymbol{\lambda} \geq \mathbf{0} \\ \mathbf{g}(\mathbf{x}) = (A^{(i)} \mathbf{x} - \mathbf{b}^{(i)}) \geq \mathbf{0} \\ \mathbf{h}(\mathbf{x}) = (A^{(e)} \mathbf{x} - \mathbf{b}^{(e)}) = \mathbf{0}. \end{cases} \quad (79)$$

### 11.3 Only Equality Constraints for QP

Let us consider a QP Problem with only equality constraints; then (78) changes into

$$\begin{cases} \min_{\mathbf{x} \in \mathbb{R}^n} \frac{1}{2} \mathbf{x}^T Q \mathbf{x} + \mathbf{c}^T \mathbf{x} \\ A\mathbf{x} - \mathbf{b} = \mathbf{0} \in \mathbb{R}^m \end{cases} \quad (m \text{ equality constraints}) \quad (80)$$

and the “remaining” KKT conditions (*changing notation* and using  $\boldsymbol{\lambda}$  for the *equality* constraints) are

$$\begin{cases} \nabla_{\mathbf{x}} \mathcal{L}(\mathbf{x}, \boldsymbol{\lambda}) = Q\mathbf{x} + (\mathbf{c} \pm A^T \boldsymbol{\lambda}) = \mathbf{0} \\ (A\mathbf{x} - \mathbf{b}) = \mathbf{0}. \end{cases} \quad (81)$$

Then, the matrix representation of system (81) is

$$\begin{bmatrix} Q & \pm A^T \\ A & \mathbf{0} \end{bmatrix} \begin{bmatrix} \mathbf{x} \\ \boldsymbol{\lambda} \end{bmatrix} = \begin{bmatrix} -\mathbf{c} \\ \mathbf{b} \end{bmatrix}. \quad (82)$$

**Observation 11.1** (Regular Points). In QP with equality constraints only,  $\mathbf{x}$  is a *regular point* (see (76)) if and only if the  $m$  rows of  $A$  are linearly independent.

### 11.4 Schur Complement Resolution

Let us consider the formulation with “+” of system (81)-(82). For the first equation of the system, it holds

$$\mathbf{x} = -Q^{-1} (\mathbf{c} + A^T \boldsymbol{\lambda}); \quad (83)$$

then, replacing  $\mathbf{x}$  in the second equation, we find  $\boldsymbol{\lambda}^*$  as solution of

$$\underbrace{(AQ^{-1}A^T)}_{\text{Schur complement}} \boldsymbol{\lambda} = -\mathbf{b} - AQ^{-1}\mathbf{c}$$

and  $\mathbf{x}^*$  as solution of

$$Q\mathbf{x} = -\mathbf{c} - A^T \boldsymbol{\lambda}^*$$

**Observation 11.2** (Properties concerning QP and Schur Complement).

- $AQ^{-1}A^T$  is symm. pos. def., assuming  $Q$  non-singular and  $A$  full rank;
- Schur Complement is convenient only if:
  1.  $Q$  easily invertible (e.g. diagonal or block-diagonal);
  2.  $m \ll n$  (since  $AQ^{-1}A^T \in \mathbb{R}^{m \times m}$ )

## 11.5 Null Space Method

Assuming that  $A$  is full-rank (i.e.  $\text{rank}(A) = m$ ), let  $Z \in \mathbb{R}^{n \times (n-m)}$  be a matrix with columns a basis of  $\text{Ker}(A)$  (i.e.  $Az = \mathbf{0} \in \mathbb{R}^m$  for each column  $\mathbf{z}$  of  $Z$ ) and let  $\hat{\mathbf{x}} \in \mathbb{R}^n$  be a particular solution of the equality constraints, i.e.  $A\hat{\mathbf{x}} = \mathbf{b}$ .

The *Null Space Method* consists in making the substitution

$$\mathbf{x} = Z\mathbf{v} + \hat{\mathbf{x}}, \quad (84)$$

in the first equation of system (81) ( $\mathbf{v}$  is the new unknown) and left-multiplying it by  $Z^T$ ; the new equation w.r.t.  $\mathbf{v}$  is

$$Z^T Q Z \mathbf{v} + Z^T Q \hat{\mathbf{x}} + \underbrace{Z^T A^T}_{\text{null matrix}} \boldsymbol{\lambda} = -Z^T \mathbf{c} \quad (85)$$

Then  $\mathbf{x}^* = Z\mathbf{v}^* + \hat{\mathbf{x}}$ , where  $\mathbf{v}^*$  is solution of

$$Z^T Q Z \mathbf{v} = -Z^T (Q \hat{\mathbf{x}} + \mathbf{c}), \quad (86)$$

and  $\boldsymbol{\lambda}^*$  is solution of

$$AA^T \boldsymbol{\lambda} = -A(\mathbf{c} + Q\mathbf{x}^*), \quad (87)$$

where (87) is obtained left-multiplying by  $A$  the first equation of system (81)

**Observation 11.3** (Properties concerning QP and Null Space Method).  $Q^{-1}$  is not needed.

### 11.5.1 Creation of $Z$

A matrix  $Z$  that satisfies the assumptions of the null space method is

$$Z = \begin{bmatrix} -A_1^{-1}A_2 \\ \mathbb{I}_{(n-m)} \end{bmatrix}, \quad (88)$$

where  $A_1 \in \mathbb{R}^{m \times m}$ ,  $A_2 \in \mathbb{R}^{m \times (n-m)}$  are such that

$$A = \begin{bmatrix} A_1 & | & A_2 \end{bmatrix}, \quad (89)$$

assuming  $A$  reordered for having  $A_1$  invertible (possible because  $A$  is full-rank); indeed, looking

for  $Z = \begin{bmatrix} Z_1 \\ \mathbb{I}_{(n-m)} \end{bmatrix}$  such that  $AZ = \mathbf{0}_{m \times (n-m)}$ , we have that

$$AZ = A_1 Z_1 + A_2 \mathbb{I}_{(n-m)}, \quad (90)$$

and then  $Z_1 = -A_1^{-1}A_2$ .

### 11.5.2 Creation of $\hat{\mathbf{x}}$

Applying the same criteria of the creation of  $Z$ , we have that the typical  $\hat{\mathbf{x}}$  is

$$\hat{\mathbf{x}} = \begin{bmatrix} A_1^{-1}(\mathbf{b} - A_2 \mathbf{x}_2) \\ \mathbf{x}_2 \end{bmatrix}, \quad (91)$$

where  $\mathbf{x}_2 \in \mathbb{R}^{n-m}$  is an arbitrary vector, e.g.

$$\hat{\mathbf{x}} = \begin{bmatrix} A_1^{-1}\mathbf{b} \\ \mathbf{0} \end{bmatrix}. \quad (92)$$

## 11.6 Exercises

The text of the exercises is for Matlab users. Nonetheless, you can use Python instead.

**Exercise 11.1** (Schur 1). Write a Matlab function `QPeq_Schur.m` that implements the Schur method for QP with equality constraints (see Section 11.4). In particular, the function takes the following inputs.

- $Q$ ,  $Q_{inv}$ : the matrix  $Q \in \mathbb{R}^{n \times n}$  of the QP loss function and its inverse;
- $c$ : the vector  $c \in \mathbb{R}^n$  of the QP loss function;
- $A$ ,  $b$ : the matrix  $A \in \mathbb{R}^{m \times n}$  and the vector  $b \in \mathbb{R}^m$  characterizing the equality constraints.

The outputs of the function must be:

- $xstar$ : solution  $x^*$  of the QP problem computed by the program;
- $fstar$ : value of the loss in the solution, i.e.  $f(x^*)$ ;
- $lambda\_star$ : lagrangian multiplier  $\lambda^*$  w.r.t.  $x^*$ ;
- $KKT\_gradL\_norm$ : norm of  $\nabla_x \mathcal{L}(x^*, \lambda^*)$ ;
- $KKT\_eq\_norm$ : norm of  $Ax^* - b$ .

**Exercise 11.2** (Schur 2). Write a Matlab function `QPeq_Schur_autoinv.m` that implements the Schur method for QP with equality constraints as in the previous exercise but without giving as input the matrix  $Q_{inv}$  and **without using the `inv` Matlab function** in the code.

**Exercise 11.3** (Null Space). Write a Matlab function `QPeq_Null.m` that implement the Null Space method for QP with equality constraints (see Appendix 11.5). In particular, the function takes the following inputs.

- $Q$ ,  $c$ : the matrix  $Q \in \mathbb{R}^{n \times n}$  of the QP loss function and the vector  $c \in \mathbb{R}^n$ ;
- $A$ ,  $b$ : the matrix  $A \in \mathbb{R}^{m \times n}$  and the vector  $b \in \mathbb{R}^m$  characterizing the equality constraints.
- $x2$ : arbitrary vector  $x_2 \in \mathbb{R}^{n-m}$  for the initialization of  $\hat{x}$  (see (91));

The outputs of the function must be:

- $xstar$ : solution  $x^*$  of the QP problem computed by the program;
- $fstar$ : value of the loss in the solution, i.e.  $f(x^*)$ ;
- $lambda\_star$ : lagrangian multiplier  $\lambda^*$  w.r.t.  $x^*$ .
- $v\_star$ : the vector  $v^*$  such that  $x^* = Zv^* + x^*$ ;
- $KKT\_gradL\_norm$ : norm of  $\nabla_x \mathcal{L}(x^*, \lambda^*)$ ;
- $KKT\_eq\_norm$ : norm of  $Ax^* - b$ .

**Exercise 11.4** (Tests). Test the functions written during the previous exercises on the following cases:

- $n = 50$ ,  $m = 10$ ;  $A = \text{rand}(m, n)$  (check the full rank),  $b = \text{rand}(m, 1)$ ,  $c = \text{rand}(n, 1)$ ;  $Q$  random positive definite diagonal matrix (use `diag` function).
- $n = 50$ ,  $m = 10$ ;  $A = \text{rand}(m, n)$  (check the full rank),  $b = \text{rand}(m, 1)$ ,  $c = \text{rand}(n, 1)$ ;  $Q = BB^T + I$  random symmetric positive definite block-diagonal matrix ( $k \geq 3$  blocks of arbitrary size), where  $B$  is a random  $k$ -block-diagonal matrix (use `blkdiag`).
- $n = 50$ ,  $m = 30$ ;  $A = \text{rand}(m, n)$  (check the full rank),  $b = \text{rand}(m, 1)$ ,  $c = \text{rand}(n, 1)$ ;  $Q = BB^T + I$  random symmetric positive definite matrix, where  $B$  is a random  $n$ -by- $n$  matrix.



## 12 Automatic Differentiation

*Automatic differentiation* (AD) is a numerical method to compute the derivatives of a function, representing it as an “augmented” computer program made of a composition of elementary operations for which the derivatives are known; typically, this program is represented as a graph connecting the elementary operations.

Then, the overall derivative is computed by exploiting the chain rule of calculus:

$$(f \circ g)' = (f' \circ g) \cdot g' \quad \text{and} \quad \frac{d}{dx}(f \circ g)(x) = \frac{d}{dx}f(g(x)) = f'(g(x)) \cdot g'(x), \quad (93)$$

also written<sup>8</sup> as

$$\frac{\partial z}{\partial x} = \frac{\partial z}{\partial y} \frac{\partial y}{\partial x} \quad \text{and} \quad \left. \frac{\partial z}{\partial x} \right|_x = \left. \frac{\partial z}{\partial y} \right|_{g(x)} \left. \frac{\partial y}{\partial x} \right|_x, \quad (94)$$

where

$$\begin{array}{ccc} f : \mathbb{R} \rightarrow \mathbb{R} & & g : \mathbb{R} \rightarrow \mathbb{R} \\ y \mapsto z & \text{and} & x \mapsto y \end{array}. \quad (95)$$

The first idea about AD dates back to the 1950s and the “forward mode AD” was developed in the 1960s. In the 1970s, Linnainmaa developed the “reverse mode AD”. This method was equivalent to the one independently developed by NN researchers in the late 1980s, with the name of backpropagation. Nowadays, backpropagation is recognized as the application of the reverse mode AD for derivatives computation in NNs.

### 12.1 Computational Graph

Let us consider an example with a function  $\ell : \mathbb{R}^2 \rightarrow \mathbb{R}$  with expression

$$\ell(x_1, x_2) = \log x_1 + x_1 x_2 - \log(\sin x_2); \quad (96)$$

then,  $\ell$  can be defined through a composition of elementary operations, represented as in Figure 5. In the graph representation of  $\ell$ , let  $y_1, y_2$  denote the input variables  $x_1, x_2$ , respectively. The intermediate variables are denoted by  $y_i$ , the output variable by  $z$ :

- $y_1 = x_1; \quad y_2 = x_2;$
- $y_3 = \log(y_1); \quad y_4 = y_1 y_2; \quad y_5 = \sin y_2;$
- $y_6 = y_3 + y_4; \quad y_7 = \log y_5;$
- $y_8 = y_6 - y_7;$
- $z = y_8.$

### 12.2 Automatic Differentiation: Forward Mode

The forward accumulation mode of AD is the conceptually most simple one since it is a recursive application of the chain rule (93). More specifically, let  $\ell : \mathbb{R}^n \rightarrow \mathbb{R}$  be a function with input variables denoted by  $\mathbf{x} = (x_1, \dots, x_n) \in \mathbb{R}^n$ ;  $y_i = x_i$ , for each  $i = 1, \dots, n$ , and  $y_i$  intermediate variable, for each  $i > n$ . Let  $x_j \in \{x_1, \dots, x_n\}$  be an input variable and  $\dot{y}_i := \partial y_i / \partial x_j$ . Then, it holds that

$$\dot{y}_i = \begin{cases} \delta_{ij}, & \forall i = 1, \dots, n \\ \sum_{y_k \text{ parent } y_i} \frac{\partial y_i}{\partial y_k} \dot{y}_k, & \forall i > n \end{cases}. \quad (97)$$

If we consider the example function  $\ell$  (see (96)), we can build another “composition graph” having the same connectivity of Figure 5 but elementary operations given by (97) (see Figure 6).

Then, given a vector  $\mathbf{x}^* \in \mathbb{R}^2$ , we can compute the value of the derivative  $\ell_{x_j}(\mathbf{x}^*)$  with the following steps:

---

<sup>8</sup>Liebniz notation.

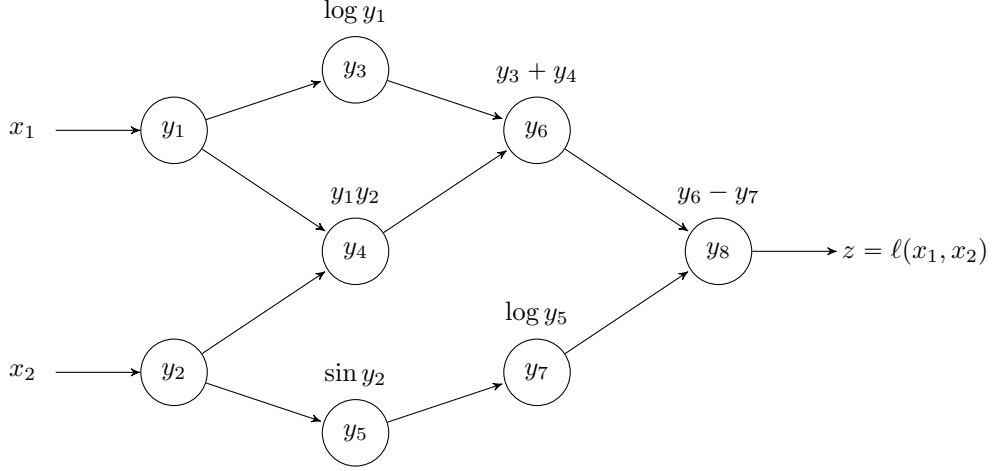


Figure 5: Graph representing  $\ell(x_1, x_2) = \log x_1 + x_1 x_2 - \log(\sin x_2)$  as a composition of elementary functions.

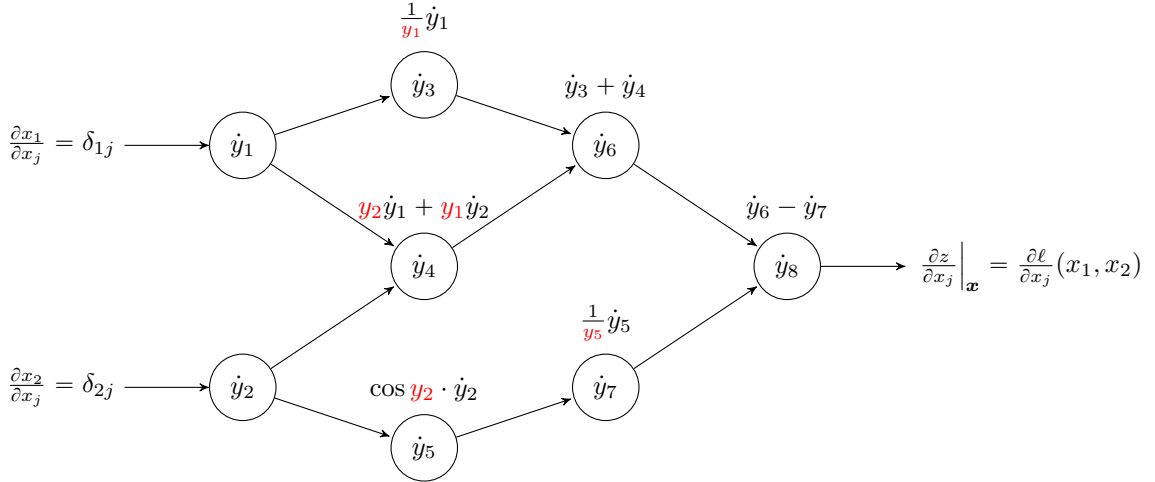


Figure 6: Graph representing  $\ell_{x_j}(x_1, x_2)$  as a composition of elementary functions, derived from the graph in Figure 5. The variables in red are computed running the operations of Figure 5 with respect to  $\mathbf{x} = (x_1, x_2)$ .

1. We compute  $\ell(\mathbf{x}^*)$  and we store all the intermediate variable values involved in the computations of the second graph (red variables in Figure 6);
2. We compute the output value of the function represented by the graph of Figure 6, giving as input (a.k.a. *seed*) the canonical basis vector  $\mathbf{e}_j$  (i.e., selecting  $x_j$  as input variable for the partial derivatives).

We observe that the forward mode AD can be easily extended to vectorial functions  $\mathbf{L} : \mathbb{R}^n \rightarrow \mathbb{R}^m$ . In particular, given a vector  $\mathbf{x}^* \in \mathbb{R}^n$ , the Jacobian matrix  $J\mathbf{L}(\mathbf{x}^*) \in \mathbb{R}^{m \times n}$  at  $\mathbf{x}^*$  is computed running  $n$  times the forward mode AD, using the  $n$  canonical basis vectors  $\mathbf{e}_1, \dots, \mathbf{e}_n \in \mathbb{R}^n$  as seeds to the “forward differentiation graph”. Indeed, independently of the output dimension  $m$ , the differentiation graph with input vector  $\mathbf{e}_j$  returns the  $j$ -th column of the Jacobian matrix  $J\mathbf{L}(\mathbf{x}^*)$ .

### 12.3 Automatic Differentiation: Reverse Mode

The reverse accumulation mode of AD reads “backwardly” the connections of the composition graph of the function to exploit the differentiation dependencies. This procedure is some-how equivalent to “explode” the partial derivatives  $\partial z / \partial x_j$  and identifying their elementary functions. Let

$$\tilde{y}_i := \sum_{y_k \text{ child of } y_i} \frac{\partial y_k}{\partial y_i} \tilde{y}_k \quad \text{and} \quad \tilde{z} := 1. \quad (98)$$

Then, for each  $i = 1, \dots, n$ , it holds

$$\tilde{y}_i = \frac{\partial z}{\partial x_i}.$$

It is easy to observe that the differentiation dependencies written in (98) are evident when reading the graph in Figure 5 from right to left. Then, the reverse mode AD consists of building another composition graph having the same connectivity but executed in the opposite way and where the elementary operations are given by (98). If we consider the example function  $\ell$  (see (96)), we can build this “reverse composition graph” (see Figure 7).

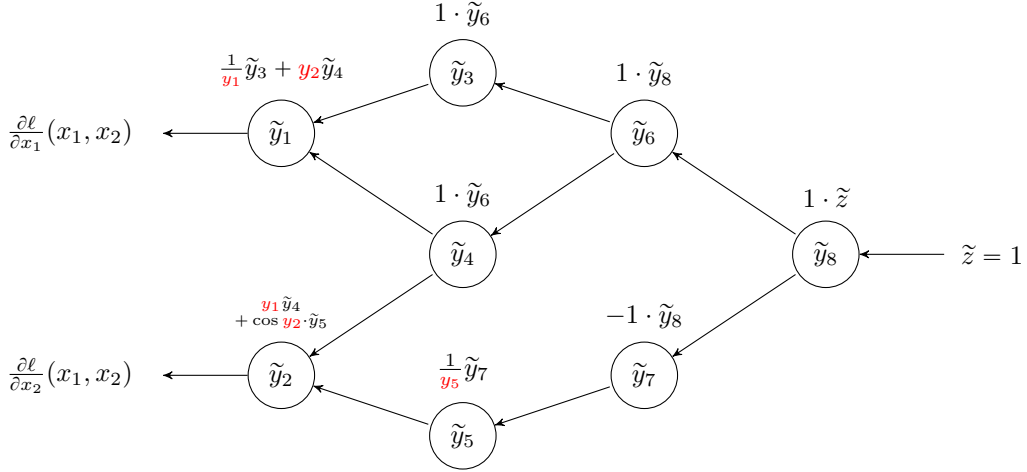


Figure 7: Graph representing  $\nabla \ell(x_1, x_2)$  as a composition of elementary functions, derived from the graph in Figure 5. The variables in red are computed running the operations of Figure 5 with respect to  $\mathbf{x} = (x_1, x_2)$ .

Given a vector  $\mathbf{x}^* \in \mathbb{R}^2$ , we can compute the value of the entire gradient  $\nabla \ell(\mathbf{x}^*)$  with the following steps:

1. We compute  $\ell(\mathbf{x}^*)$  and we store all the intermediate variable values involved in the computations of the second graph (red variables in Figure 7);
2. We compute the output values of the function represented by the graph of Figure 7, giving as seed the scalar value equal to 1 (i.e.,  $\tilde{z} = 1$ ).

We observe that the reverse mode AD can be easily extended to vectorial functions  $\mathbf{L} : \mathbb{R}^n \rightarrow \mathbb{R}^m$ . In particular, given a vector  $\mathbf{x}^* \in \mathbb{R}^n$ , the Jacobian matrix  $J\mathbf{L}(\mathbf{x}^*) \in \mathbb{R}^{m \times n}$  at  $\mathbf{x}^*$  is computed running  $m$  times the reverse mode AD, using the  $m$  canonical basis vectors  $\mathbf{e}_1, \dots, \mathbf{e}_m \in \mathbb{R}^m$  as seeds to the differentiation graph. Indeed, independently of the input dimension  $n$ , the differentiation graph with input vector  $\mathbf{e}_j$  returns the  $j$ -th row of the Jacobian matrix  $J\mathbf{L}(\mathbf{x}^*)$ .

## 12.4 Reverse AD in Matlab

In Matlab<sup>9</sup>, the *Deep Learning toolbox* let us use the reverse AD to computed derivatives of functions (defined by elementary operations). There are three main matlab functions for the reverse AD:

`x_ad = dlarray(x)`: in Matlab, AD does not work with “classic” matlab arrays. Then, this function changes the array type into the correct one. To convert a *dlarray* into a “classic” array, use the `extractdata` function.

`[dydx1, ..., dydxn] = dlgradient(y,x1,...,xk)`: given a scalar value  $y$ , obtained by the composition of elementary operations that involve the variables  $x_1, \dots, x_k$ , this function “prepares” the graph for the gradient computation of  $y$ . This function must be called inside a function or inside a function handle;

`dlfeval(ydlgradf, x_ad)` or `dlfeval(@ydlgradf, x_ad)`: this function calls a function handle or a function<sup>10</sup> `ydlgradf` containing `dlgradient`, using as declared variables the values inside `x_ad`.

## 12.5 Exercises

**Exercise 12.1** (AD Implementation). *Look at the section above and at this page (<https://it.mathworks.com/help/deeplearning/ref/dlarray.dlgradient.html>); then, write a matlab function and/or a matlab script that, given*

$$f(\mathbf{x}) = \sin\left(\frac{\pi}{4}\|\mathbf{x}\|^2\right), \quad (99)$$

*compute its gradient.*

**Exercise 12.2** (AD and Steepest Descent). *Perform the exercise of the steepest descent laboratory, using AD to define the gradient function.*

**Suggestion:** the `extractdata` function can be useful.

**Exercise 12.3** (AD and Higher Order Derivatives - Homework). *Activate the option 'EnableHigherDerivatives' in `dlgradient` and modify the previous exercise to compute also the Hessian of  $f$ .*

**Remember:** `dlgradient` works only with scalar values for  $y$ . However, latest Matlab versions have further AD tools, e.g.: `dljacobian`.

---

<sup>9</sup>Most of programming languages has frameworks for AD computation. E.g., in python there are TensorFlow and PyTorch.

<sup>10</sup>in the case of a function, the symbol @ is necessary.

## A Material for Refreshing Basic Knowledge

**MATLAB pills:** If you want to use MATLAB and are not so familiar with it, you can find it useful to follow these video pills:

1. Introduction: [https://youtu.be/J2-R\\_Hw4Ak8](https://youtu.be/J2-R_Hw4Ak8)
2. Variables and Scripts: <https://youtu.be/oI4ZP-GxEuU>
3. Functions: [https://youtu.be/C\\_59NdVeD0Q](https://youtu.be/C_59NdVeD0Q)
4. Arrays and Matrices (part 1): <https://youtu.be/17TClaukIkE>
5. Indexing - Accessing Elements in Arrays and Matrices: <https://youtu.be/OB1I3FJn9Sg>
6. Operations with Arrays and Matrices: <https://youtu.be/J-gzenf-4bY>
7. Arrays and Matrices (part 2): <https://youtu.be/e9CdE95va-g>
8. Logical Indexing: [https://youtu.be/\\_vjDMKF9zcQ](https://youtu.be/_vjDMKF9zcQ)

**Background (Numerical Methods for Linear Algebra):** if you need to consolidate your background in basic numerical methods for linear algebra, you may find it helpful to watch the following video lectures: from Portale della Didattica (general part, not the course page!) follow **Materiale** -> **Lezioni on-line** -> **Primo Anno** -> **Linear Algebra and Geometry**.

The suggested lectures are those delivered by Prof. Dabbene, and should be enough to watch lectures 5, 9, 13 (neglect the part on polynomials), 22, 29, and 33. Italian students may prefer to watch the lectures in Italian, then just look for Algebra Lineare e Geometria and look at lectures from Prof.ssa Scuderi (lectures 4, 8, 12, 24 [ignore the initial part on approximation], 28, 30).

## B Test Functions - Optimization

When you have to test an optimization method, it is useful to use some “standard” test functions<sup>11</sup>. Here we present some test functions that will be used in the laboratories.

### B.1 The Rosenbrock and the Himmelblau Test Functions

In this section we introduce the the 2-dimensional *Rosenbrock* function and the *Himmelblau* function that are the functions  $f_2$  and  $f_3$ , respectively, in the file `test_functions2.mat`. The other function, denoted as  $f_1$ , is instead a simple paraboloid.

- **Paraboloid** ( $n = 2$ ):

$$\begin{aligned} f_1(x_1, x_2) &= x_1^2 + 4x_2^2 + 5 \\ \nabla f_1(x_1, x_2) &= \begin{bmatrix} 2x_1 \\ 8x_2 \end{bmatrix} \\ H_{f_1}(x_1, x_2) &= \begin{bmatrix} 2 & 0 \\ 0 & 8 \end{bmatrix} \quad (constant) \\ \text{global minima} &= \{f_1(0, 0) = 5\} \\ \text{test } \mathbf{x}^{(0)} &= \{(0, 0), (5, 0)\} \end{aligned} \tag{100}$$

---

<sup>11</sup>E.g., see [https://en.wikipedia.org/wiki/Test\\_functions\\_for\\_optimization](https://en.wikipedia.org/wiki/Test_functions_for_optimization)

- **Rosenbrock** ( $n = 2$ ):

$$\begin{aligned}
f_2(x_1, x_2) &= 100(x_2 - x_1^2)^2 + (1 - x_1)^2 \\
\nabla f_2(x_1, x_2) &= \begin{bmatrix} 400x_1^3 - 400x_1x_2 + 2x_1 - 2 \\ 200(x_2 - x_1^2) \end{bmatrix} \\
H_{f_2}(x_1, x_2) &= \begin{bmatrix} 1200x_1^2 - 400x_2 + 2 & -400x_1 \\ -400x_1 & 200 \end{bmatrix} \\
\text{global minima} &= \{f_2(1, 1) = 0\} \\
\text{test } \mathbf{x}^{(0)} &= \{(1.2, 1.2), (-1.2, 1)\}
\end{aligned} \tag{101}$$

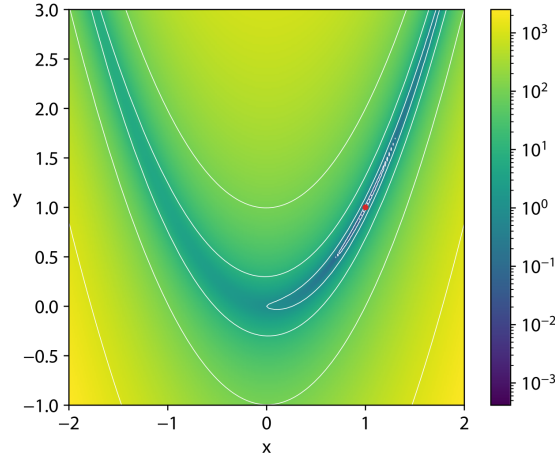


Figure 8: Rosenbrock function ( $n = 2$ ), top view

- **Himmelblau**:

$$\begin{aligned}
f_3(x_1, x_2) &= (x_1^2 + x_2 - 11)^2 + (x_1 + x_2^2 - 7)^2 \\
\nabla f_3(x_1, x_2) &= \begin{bmatrix} 4x_1^3 + 4x_1x_2 - 42x_1 + 2x_2^2 - 14 \\ 4x_2^3 + 4x_1x_2 - 26x_2 + 2x_1^2 - 22 \end{bmatrix} \\
H_{f_3}(x_1, x_2) &= \begin{bmatrix} 12x_1^2 + 4x_2 - 42 & 4x_1 + 4x_2 \\ 4x_1 + 4x_2 & 12x_2^2 + 4x_1 - 26 \end{bmatrix} \\
\text{global minima} &= \left\{ f_3(\mathbf{x}^{*(i)}) = 0 \mid i = 1, \dots, 4 \right\}, \quad \text{where} \\
\mathbf{x}^{*(1)} &= (3, 2), \quad \mathbf{x}^{*(2)} \simeq (-2.805118, 3.131312) \\
\mathbf{x}^{*(3)} &\simeq (-3.779310, -3.283186), \quad \mathbf{x}^{*(4)} \simeq (3.584428, -1.848126) \\
\text{test } \mathbf{x}^{(0)} &= \{(0, 0), \text{random in } [-4, 4]^2\}
\end{aligned} \tag{102}$$

## B.2 $n$ -dimensional Test Function

In this section, we introduce the  $n$ -dimensional, parametric, *Rosenbrock* function. See the 2-dimensional version in Section B.1.

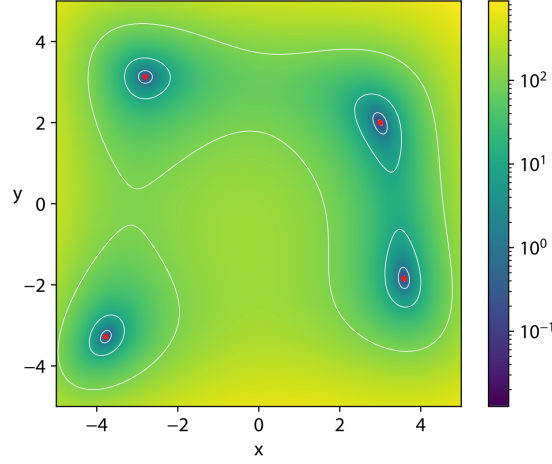


Figure 9: Himmelblau function, top view

- **Parametric Rosenbrock** ( $\alpha > 0$ ,  $n \in \mathbb{N}$ ):

$$\begin{aligned}
 f(\mathbf{x}) &= \sum_{i=1}^{n-1} (\alpha(x_{i+1} - x_i^2)^2 + (1 - x_i)^2) \\
 \nabla f(\mathbf{x}) &= \dots \text{exercise} \dots \\
 Hf(\mathbf{x}) &= \dots \text{exercise} \dots \\
 \text{global minima} &= \{f(1, \dots, 1) = 0\} \\
 \text{test } \mathbf{x}^{(0)} &= \{(1.2, \dots, 1.2), (-1.2, 1, \dots, -1.2, 1)\} \quad (\text{assuming } n \text{ even})
 \end{aligned} \tag{103}$$

**Hints for computing derivatives:**

$$\frac{\partial}{\partial x_i} f(\mathbf{x}) = \frac{\partial}{\partial x_i} (\alpha(x_{i+1} - x_i^2)^2 + (1 - x_i)^2 + \alpha(x_i - x_{i-1}^2)^2 + (1 - x_{i-1})^2), \tag{104}$$

for each  $i = 2, \dots, n-1$ ; for  $i = 1, n$ , small changes occur:

$$\begin{aligned}
 \frac{\partial}{\partial x_1} f(\mathbf{x}) &= \frac{\partial}{\partial x_1} (\alpha(x_2 - x_1^2)^2 + (1 - x_1)^2) \\
 \frac{\partial}{\partial x_n} f(\mathbf{x}) &= \frac{\partial}{\partial x_n} (\alpha(x_n - x_{n-1}^2)^2 + (1 - x_{n-1})^2),
 \end{aligned} \tag{105}$$

Moreover, you can generalize (104) to the case of second order derivatives for computing the Hessian, observing also that most of its elements are zeros; specifically, the Hessian is tridiagonal and symmetric (see Figure 10):

$$\text{if } j < (i-1) \text{ or } j > (i+1) \quad \Rightarrow \quad \frac{\partial^2}{\partial x_i \partial x_j} f(\mathbf{x}) = 0. \tag{106}$$

Both the gradient and the Hessian of the function can be written directly as a function handle; nonetheless, if you prefer to have a more readable code, you can write them as a Matlab function instead (loading them as function handle variables just for running the optimization functions).

**Hints for computing Finite Differences:**

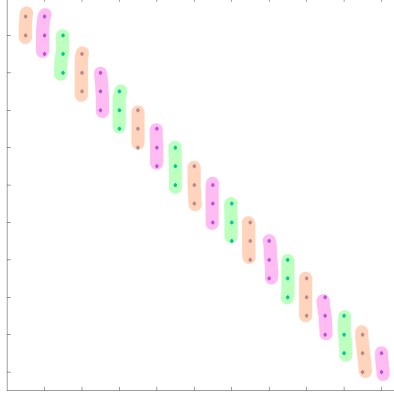


Figure 10: Scheme of the Hessian for the  $n$ -dimensional parametric Rosenbrock function. Colors show an example of exploiting the sparsity of the Hessian if approximated as the Jacobian of the exact gradient.

Let  $f_1 : \mathbb{R}^2 \rightarrow \mathbb{R}$  and  $f_2 : \mathbb{R} \rightarrow \mathbb{R}$  be such that  $f_1(y_1, y_2) = \alpha(y_1 - y_2^2)^2$  and  $f_2(y) = (1 - y)^2$ . Therefore, we have that

$$f(\mathbf{x}) = \sum_{i=1}^{n-1} (\alpha(x_{i+1} - x_i^2)^2 + (1 - x_i)^2) = \sum_{i=1}^{n-1} (f_1(x_{i+1}, x_i) + f_2(x_i)) \quad (107)$$

and

$$\begin{aligned} f(\hat{\mathbf{x}} + h\mathbf{e}_i) &= (f_1(\hat{x}_2, \hat{x}_1)) + f_2(\hat{x}_1) + \dots + \\ &\quad + (f_1(\hat{x}_i + h, \hat{x}_{i-1}) + f_2(\hat{x}_{i-1})) + (f_1(\hat{x}_{i+1}, \hat{x}_i + h) + f_2(\hat{x}_i + h)) + \dots + \\ &\quad + (f_1(\hat{x}_n, \hat{x}_{n-1}) + f_2(\hat{x}_{n-1})), \quad \forall i = 2, \dots, n-1, \end{aligned}$$

$$\begin{aligned} f(\hat{\mathbf{x}} + h\mathbf{e}_1) &= (f_1(\hat{x}_2, \hat{x}_1 + h)) + f_2(\hat{x}_1 + h) + \dots + \\ &\quad + (f_1(\hat{x}_n, \hat{x}_{n-1}) + f_2(\hat{x}_{n-1})), \end{aligned} \quad (108)$$

$$\begin{aligned} f(\hat{\mathbf{x}} + h\mathbf{e}_n) &= (f_1(\hat{x}_2, \hat{x}_1)) + f_2(\hat{x}_1) + \dots + \\ &\quad + (f_1(\hat{x}_n + h, \hat{x}_{n-1}) + f_2(\hat{x}_{n-1})). \end{aligned}$$

Then, for example:

$$\begin{aligned} f(\hat{\mathbf{x}} + h\mathbf{e}_i) - f(\hat{\mathbf{x}}) &= f_1(\hat{x}_i + h, \hat{x}_{i-1}) + (f_1(\hat{x}_{i+1}, \hat{x}_i + h) + f_2(\hat{x}_i + h)) - \\ &\quad - (f_1(\hat{x}_i, \hat{x}_{i-1}) + (f_1(\hat{x}_{i+1}, \hat{x}_i) + f_2(\hat{x}_i))), \quad \forall i = 2, \dots, n-1, \end{aligned}$$

$$f(\hat{\mathbf{x}} + h\mathbf{e}_1) - f(\hat{\mathbf{x}}) = (f_1(\hat{x}_2, \hat{x}_1 + h)) + f_2(\hat{x}_1 + h) - (f_1(\hat{x}_2, \hat{x}_1) + f_2(\hat{x}_1)), \quad (109)$$

$$f(\hat{\mathbf{x}} + h\mathbf{e}_n) - f(\hat{\mathbf{x}}) = f_1(\hat{x}_n + h, \hat{x}_{n-1}) - f_1(\hat{x}_n, \hat{x}_{n-1}).$$

**N.B.:** all these observations can be generalized to the diagonals of the Hessian matrix (see Figure 10) if computed with second order FD (not as approximated Jacobian of the exact gradient).

## C Test Functions - Equations and Systems

In this section, we list some test functions for nonlinear equations and systems.



## C.1 Nonlinear Equations

We introduce the following test functions of the type  $\mathbf{F} : \mathbb{R}^n \rightarrow \mathbb{R}^n$ :

- $\mathbf{F} : \mathbb{R}^2 \rightarrow \mathbb{R}^2$  such that

$$\mathbf{F}(\mathbf{x}) = \begin{bmatrix} f_1(\mathbf{x}) \\ f_2(\mathbf{x}) \end{bmatrix} = \begin{bmatrix} 2x_1 - x_2 - e^{-x_1} \\ 2x_2 - x_1 - e^{-x_2} \end{bmatrix}, \quad (110)$$

and with jacobian

$$J\mathbf{F}(\mathbf{x}) = \begin{bmatrix} 2e^{-x_1} & -1 \\ -1 & 2e^{-x_2} \end{bmatrix}. \quad (111)$$

- $\mathbf{G} : \mathbb{R}^2 \rightarrow \mathbb{R}^2$  such that

$$\mathbf{G}(\mathbf{x}) = \begin{bmatrix} g_1(\mathbf{x}) \\ g_2(\mathbf{x}) \end{bmatrix} = \begin{bmatrix} x_2^2 - 1 \\ \sin x_1 - x_2 \end{bmatrix}, \quad (112)$$

and with jacobian

$$J\mathbf{G}(\mathbf{x}) = \begin{bmatrix} 0 & 2x_2 \\ \cos x_1 & -1 \end{bmatrix}. \quad (113)$$

**N.B.:** These functions are stored in the file `test_nonlinsys.mat`.

## References

- [1] Jorge Nocedal and Stephen J. Wright. *Numerical optimization*. Second. 9781447122234. Springer, 2012. ISBN: 9780387303031. DOI: [10.1007/978-1-4471-2224-1\\_2](https://doi.org/10.1007/978-1-4471-2224-1_2).
- [2] Sandra Pieraccini. “Numerical Optimization for Large Scale Problems and Stochastic Optimization”. In: *Draft slides* (2023).